CRAFT: COMPOSITIONAL REWARD ALIGNMENT FOR TARGET-HIDDEN LOOP VERIFICATION

Anonymous authors
Paper under double-blind review

ABSTRACT

Valid loop invariants are closed under conjunction: useful clauses from different responses can prove a target even when no response does so alone. Yet standard training scores each response as an indivisible answer. We study this mismatch under *target-hidden loop verification*, where target assertions and postconditions Q are withheld during generation and training. CRAFT aligns both stages through rollout–decompose–pool–filter–compose. At inference, it decomposes responses into clauses, pools them across responses, filters the pool with syntax checking and Houdini, and composes the survivors into one invariant before restoring Q. SFT distills a filtered multi-response union into one response. RL assigns credit within a rollout to its verifier-retained subset and across rollouts to complementary coverage beyond peers. Its target-independent strength signal comes from reachable executions and conservatively filtered synthetic negatives. On 832 C programs, four `gpt-5-nano` rollouts verify 49.0% of tasks, compared with 42.2% for the strongest external baseline in an end-to-end target-hidden evaluation with system-native fixed budgets.

1 INTRODUCTION

Loop invariants turn unbounded executions into finite proofs, but a formally valid invariant can still be useless. An invariant is *valid* if it holds when execution first reaches the loop and remains true after every loop iteration. A verifier can check these obligations, yet its pass/fail outcome provides no graded measure of how much the invariant says: $true$ is valid for every loop but proves nothing about its behavior or desired outcome. A useful invariant must also be strong enough to establish downstream properties. Since the exact reachable-state set may be expensive to compute or inexpressible in the annotation language, generation must navigate many valid but weak over-approximations and receive credit for approaching stronger ones.

This search need not succeed in a single attempt. Validity is closed under conjunction: the conjunction of two valid invariants is valid and no weaker than either one. Useful clauses can therefore be accumulated across samples instead of demanding a complete answer from one response. Model responses expose such partial results as clauses: one may discover a range bound, another a relational equality, and together they may prove a target that neither proves alone. Clause-wise verification can discard invalid clauses while preserving valid ones, motivating the filtered clause pool as the inference object.

Response-level rewards are mismatched to this unit in two ways. First, verifiable rewards credited to whole responses chiefly sharpen the sampling distribution toward responses the policy already produces, improving pass@1 without widening what k samples can cover (Yue et al., 2025). For a clause pool, such sharpening toward a few individually complete responses is at best orthogonal and at worst collapses the diversity that composition exploits. Second, a complete response is also the wrong unit of training credit. A whole-response reward withholds credit from useful clauses whenever another clause in the same response fails. A reward computed independently for each response cannot distinguish a constraint that expands group coverage from one already repeated by every peer. Under repeated sampling, this can encourage the policy to reproduce easy high-reward clauses instead of exploring a clause pool whose members work together. Our central claim is that loop-invariant learning should optimize the verified composition obtained from a fixed rollout group, rather than treating every rollout as an independent final answer.

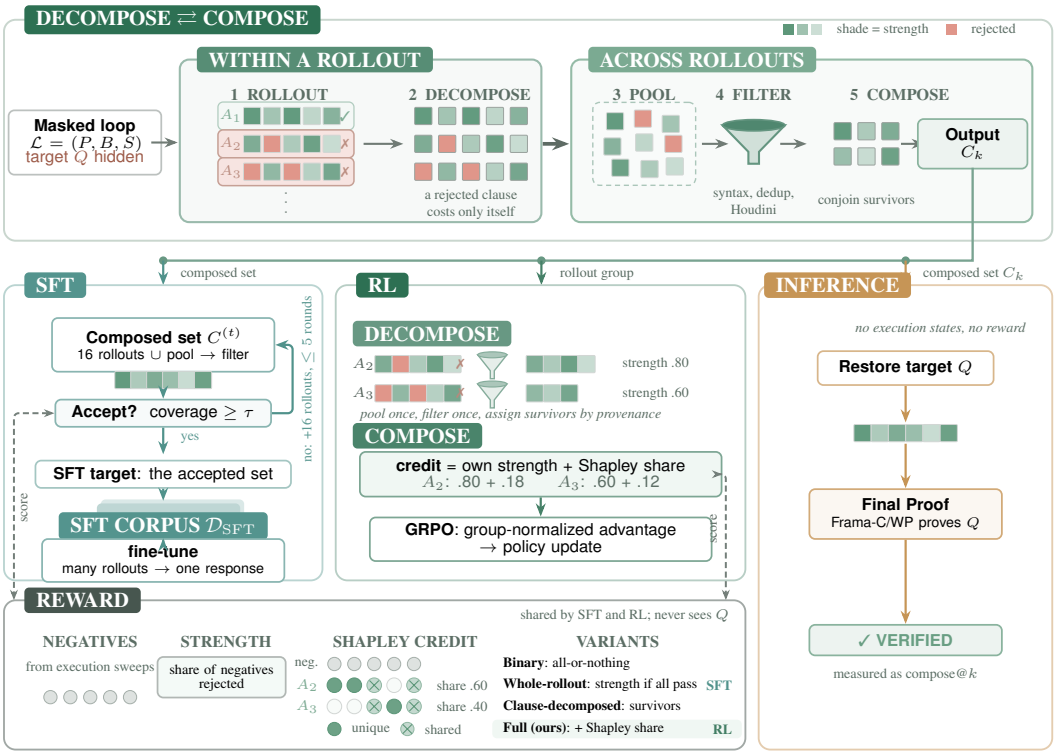

Figure 1: **Decompose–compose is the organizing principle.** Each target-hidden rollout is *decomposed* into clauses that stand alone, and their union is *composed* into one filtered invariant; the composed set feeds three consumers. One target-independent *reward* scores clause sets against execution-derived negatives and is shared by two of them: SFT accepts a composed set once its whole-rollout coverage reaches τ, strengthening it with further rollout groups otherwise, and distills it into one SFT response; RL *decomposes* each rollout onto the pooled survivor set back onto each rollout and credits it with clause-level coverage plus a closed-form Shapley share of the group's rejections, so a rejection already supplied by peers earns less. The four credit rules ablated in Section 5.6 are listed in the reward panel. Inference never touches executions or the reward: it restores Q only for the final proof.

We study this claim under *target-hidden loop verification*. The model sees the function preconditions, executable prefix, loop guard, and body, but the target set Q is withheld until the generated invariant set has been fixed. The final test still uses the original task: after Q is restored, the verifier must establish the loop obligations and prove every target at its original program point. Hiding Q prevents a model from copying a target that happens to be valid or using target-specific proof failures to steer generation. It also rules out target success as a training reward, making a graded, target-independent measure of invariant strength necessary. The invariants this setting rewards are also the reusable ones: an invariant strong enough without knowing Q serves every property later asserted about the loop.

CRAFT organizes inference around *rollout–decompose–pool–filter–compose*. Given a budget of k target-hidden responses, the pipeline extracts each invariant annotation as a clause, unions clauses across responses, removes syntax failures, and applies Houdini to retain a jointly valid subset. Only then is Q restored for the final proof. We measure this inference-time object with compose@k: the fraction of programs verified after composing the first k responses. Unlike pass@k, which requires at least one response to contain the complete proof, compose@k preserves useful partial answers. The objective is to make this curve both higher and faster to converge, so a small rollout budget recovers the compositions otherwise found only through much broader sampling.

Training aligns credit with the two levels of this inference procedure. *Within* each rollout, we project its extracted clauses onto a verifier-retained subset; one rejected clause does not erase credit earned by the others. We score the strength of that subset using the execution-derived negative examples it

rejects, without adding a reward merely for well-formed syntax or whole-response validity. *Across* rollouts, we apply a closed-form Shapley allocation to the union of their pooled-filtered, provenance-assigned negative-example coverage: repeated rejections share credit, whereas a rejection supplied by fewer peers receives more. Thus the reward of one rollout depends on what the other rollouts in its Group Relative Policy Optimization (GRPO) group discover (Shao et al., 2024). Synthetic supervised fine-tuning (SFT) additionally distills a filtered multi-response union into one individual response. Together, these stages make sampled responses more useful to the clause pool that the pipeline composes at inference.

Prior systems synthesize and prune candidate invariants at inference time, while learning-based methods train from verified outputs, solver feedback, or behavioral examples (Flanagan & Leino, 2001; Kamath et al., 2023; Cao et al., 2025; Si et al., 2020; Yan et al., 2025; Yang et al., 2026a; Huang et al., 2026). We instead train for the filtered clause pool used at inference: credit preserves useful clauses within a response and depends on complementary coverage across responses.

Our experiments deliberately separate standalone success from composed inference. On 832 integer C programs, four target-hidden `gpt-5-nano` responses with union and Houdini verify 408 programs (49.0%), compared with 351 (42.2%) for the strongest external baseline under each system's fixed budget.

Contributions. This paper makes three contributions:

- We formulate target-hidden loop verification around a fixed-budget rollout–decompose–pool–filter–compose objective. The evaluated object at inference is the verified composition measured by compose@k, not an isolated response.
- We align learning credit with that object at two levels, without using Q. Within a rollout, the unit of credit is the verifier-retained clause, so an imperfect response is paid for what survives; across rollouts, a closed-form Shapley allocation pays complementary coverage rather than repetition.
- We show that clause-level credit is what turns RL from sharpening into expanding compose@k, that group credit raises the curve further at every budget beyond one response, and that the trained policy reaches its untrained initialization's 32-response composition with four responses. We audit the execution-derived examples that provide the graded strength signal.

2 PRELIMINARIES

Loop verification. A task consists of a loop $\mathcal{L} = (P, B, S)$ and a target set $Q = \{(\ell, q_\ell)\}$, where predicate q_ℓ occurs at assertion or postcondition location ℓ. Here P describes the loop-entry states induced by the function preconditions and executable prefix, B is the guard, and S is the body. Let $\mathrm{Reach}(\mathcal{L})$ be the reachable loop-head states, including the final guard test. An invariant I is *valid* (or inductive) when

$$P \Rightarrow I, \qquad \{I \wedge B\}\, S\, \{I\},$$

which ensures $\mathrm{Reach}(\mathcal{L}) \subseteq [\![I]\!]$ (Floyd, 1967; Hoare, 1969). The semantic ideal—an invariant whose meaning is exactly $\mathrm{Reach}(\mathcal{L})$—need not be finitely expressible in the annotation language.

For predicates I and J, write

$$I \preceq J \quad \Longleftrightarrow \quad I \Rightarrow J \quad \Longleftrightarrow \quad [\![I]\!] \subseteq [\![J]\!]\,.$$

Thus $I \preceq J$ means that I is at least as strong as J, and $I \prec J$ means that it is strictly stronger. For a finite clause set C, define

$$I_C \triangleq \bigwedge_{c \in C} c, \qquad I_\varnothing \triangleq \mathit{true}.$$

We call C *jointly inductive* when I_C is valid, equivalently when

$$P \Rightarrow I_C, \qquad \{I_C \wedge B\}\, S\, \{I_C\}.$$

We reserve $\mathsf{H}_{\mathcal{L}}(C)$ for the ideal Houdini fixed point under exact establishment and preservation decisions, and $\mathsf{Filter}_{\mathcal{L}}(C)$ for the implemented parsing, deduplication, and resource-bounded WP/Houdini pipeline.

A valid invariant is *Q-sufficient* when annotating the loop with I lets the verifier prove every q_ℓ at ℓ. For one post-loop target this reduces to $I \wedge \neg B \Rightarrow q$. Thus validity is target-independent, whereas sufficiency is tested only after restoring Q.

Target-hidden verification. The generator and all intermediate scores use $\mathcal{L} = (P, B, S)$, not Q. Assertions, postconditions, executable assertion calls, and conventional error-label names are masked; non-contract comments are removed. Preconditions and executable loop semantics remain visible. The generated invariant is fixed before the untouched program is restored for final verification. We study scalar-integer C programs with one loop; Appendix A gives the scope and a motivating nonlinear example.

3 Related Work

Candidate generation, filtering, and prompt-time composition. Classical invariant synthesis uses abstract interpretation, affine relations, templates, interpolation, recurrences, syntax-guided search, or counterexamples; execution-driven systems instead mine likely properties from traces (Cousot & Cousot, 1977; Karr, 1976; Colón et al., 2003; McMillan, 2003; Kincaid et al., 2017; Alur et al., 2013; Garg et al., 2014; Ernst et al., 2007). Houdini iteratively removes invalid candidates to retain an inductive subset (Flanagan & Leino, 2001). LLM workflows reuse this pattern: Loopy couples proposals with Houdini and repair, iRank ranks candidates, and Clause2Inv composes retained clauses while iteratively accumulating counterexamples (Kamath et al., 2023; Chakraborty et al., 2023; Cao et al., 2025). LaM4Inv, ACInv, AutoSpec, SpecGen, SESpec, LEMUR, and Baldur similarly organize generation with static analysis, symbolic execution, verification, or repair (Wu et al., 2024a; Liu et al., 2024b; Wen et al., 2024; Ma et al., 2025; Yang et al., 2026b; Wu et al., 2024b; First et al., 2023). Together, these systems establish candidate generation, verification, filtering, and repair as effective inference-time patterns.

Specialized neural invariant learning. Code2Inv learns a construction policy from solver feedback; CLN2INV and G-CLN learn continuous representations of linear and nonlinear relations; and LIPuS composes RL-based pruning with SMT solving (Si et al., 2018; 2020; Ryan et al., 2020; Yao et al., 2020; Yu et al., 2023). These task-specific models optimize an individual candidate or construction trajectory using solver-guided learning.

Behavioral-example objectives for specification strength. COINS assesses specifications using trusted positive and mutation-derived negative input–output cases (Yang et al., 2026a); SpecRL rewards complete method-level specifications that reject impossible input–output pairs (Huang et al., 2026). Both show that behavioral examples can distinguish weak but valid specifications from stronger ones. Our setting derives such examples from loop dynamics and evaluates verifier-retained subsets of imperfect responses.

Language-model training from formal feedback. InvBench and Wonda use verified data for invariant fine-tuning; CodeRL uses execution signals; and Re:Form applies verifier-guided RL to individual Dafny proofs (Wei et al., 2025; Pinto et al., 2026; Le et al., 2022; Yan et al., 2025). These methods establish verified execution or proof feedback as a training signal for code and formal-reasoning models.

Set-level credit in verifiable-reward RL. GRPO with a per-response verifiable reward tends to sharpen the base distribution: pass@1 improves while large-k coverage does not exceed the untrained model (Yue et al., 2025; He et al., 2025). Remedies transform the group's rewards so that the objective becomes a set statistic—pass@k itself (Walder & Karkhanis, 2025; Chen et al., 2025) or an up-weighting of rare correct responses (He et al., 2025). These methods keep the response as the unit of credit: a set is valued by how many of its members are individually correct. CRAFT combines these lines of work: it post-trains a general language model for the filtered clause pool used at inference, salvages verifier-retained clauses from imperfect responses, and allocates group credit according to their contribution to pooled negative-state coverage. The resulting set objective values what the responses prove jointly rather than how many responses are individually complete.

4 Method

CRAFT couples an inference protocol that proves targets by composing clauses across responses with a training pipeline that optimizes that protocol without ever seeing the target. Three design

Algorithm 1 Rollout–decompose–pool–filter–compose

Input: masked loop $\mathcal{L}$, parsed clause sequences $A_{1:G}$, cap K
Output: filtered clause set C

$C \leftarrow \bigcup_i \mathrm{set}(\mathrm{Cap}_K(\mathrm{Normalize}(A_i)))$ ▷ *decompose, pool*
$C \leftarrow \mathrm{ConservativeDedup}(C)$ ▷ *filter: dedup*
repeat ▷ *filter: syntax*
  $C_{\mathrm{parse}} \leftarrow \mathrm{ParserErrors}(\mathcal{L}, C)$; $C \leftarrow C \setminus C_{\mathrm{parse}}$
until $C_{\mathrm{parse}} = \varnothing$ or $C = \varnothing$
repeat ▷ *filter: Houdini*
  $C_{\mathrm{drop}} \leftarrow \{c \in C : \neg\, \mathrm{WP}_{\mathrm{est,pres}}(\mathcal{L}, C, c)\}$; $C \leftarrow C \setminus C_{\mathrm{drop}}$
until $C_{\mathrm{drop}} = \varnothing$ or $C = \varnothing$
return C ▷ *compose:* $I_C = \bigwedge_{c \in C} c$

decisions organize this section: (i) *target hiding*—generation, distillation, and reward computation all operate on the masked loop, and the target is restored only for the final proof (Section 4.2); (ii) *the clause, not the response, is the credit unit* that survives filtering and earns reward (Section 4.1); (iii) *training mirrors inference*—credit rewards complementary coverage across a response group, because inference unions that group (Section 4.5). The guiding thesis is that because inference composes, training should optimize composability: raising the compose@k curve and reaching its large-pool performance with fewer rollouts.

4.1 Rollout–Decompose–Pool–Filter–Compose

For a target-masked loop $\mathcal{L}$, the model emits ACSL loop-invariant annotations (Baudin et al., 2021). Parsing extracts each `loop invariant ...;` annotation as one clause; normalization turns response i into the clause sequence $A_i = \langle a_{i1}, \ldots, a_{im_i} \rangle$. We cap it at K clauses (K=20; Appendix C), $\bar{A}_i = \langle a_{i1}, \ldots, a_{i,\min(m_i,K)} \rangle$, and discard the rest. Extraction makes the annotation, rather than the complete response, the unit that can survive filtering and earn credit. The pipeline has five steps: *rollout* samples responses, *decompose* extracts their clauses, *pool* unions the clauses across responses, *filter* removes syntax failures and non-inductive clauses from the pool, and *compose* conjoins the survivors into one invariant. We write $\mathrm{set}(\bar{A}_i)$ for the underlying clause set when applying set operations.

Given G responses, the pipeline unions their clauses,

$$C_G^{\mathrm{pool}} = \bigcup_{i=1}^{G} \mathrm{set}(\bar{A}_i),$$

then iteratively removes syntax failures and applies Houdini to delete clauses whose establishment or preservation obligations fail (Flanagan & Leino, 2001). We write the resulting inductive subset as $\mathsf{Filter}_{\mathcal{L}}(C_G^{\mathrm{pool}})$. Conjoining valid clauses preserves validity and can compose a bound from one response with a relation from another; filtering keeps such clauses from otherwise imperfect responses but cannot invent a missing relation. Algorithm 1 computes $\mathsf{Filter}_{\mathcal{L}}$; we write its fixed point as $\mathsf{H}_{\mathcal{L}}(C)$ and the conjunction of a clause set C as $I_C = \bigwedge_{c \in C} c$. The algorithm is shared by SFT synthesis and inference.

The following statements formalize the composition used by the pipeline; all proofs are deferred to Appendix B.

Proposition 1 (Closure and strengthening under conjunction)**.** *Let $I_1, \ldots, I_m$ be valid invariants of the same loop, for finite $m \geq 1$. Then $I = \bigwedge_{j=1}^{m} I_j$ is valid and implies every I_j, i.e., $I \preceq I_j$. Moreover, $I \prec I_j$ exactly when $I_j \not\Rightarrow \bigwedge_{i \neq j} I_i$.*

Proposition 2 (Mutually supported composition)**.** *Let $C = \{c_1, \ldots, c_m\}$ be any finite set of candidate clauses. The conjunction I_C is a valid invariant whenever*

$$P \Rightarrow I_C, \qquad \forall j,\ \{I_C \wedge B\}\, S\, \{c_j\}.$$

Individual obligations $\{c_j \wedge B\}\, S\, \{c_j\}$ need not hold: preservation may be supplied by any number of companion clauses in C. Mutual support cannot, however, repair a clause that fails establishment from P. The resulting valid invariant satisfies $I_C \preceq c_j$, with $I_C \prec c_j$ whenever $c_j \not\Rightarrow I_{C\setminus\{c_j\}}$.

Theorem 3 (Candidate-relative optimality of ideal Houdini)**.** *For a finite parser-admitted pool C, assume exact decisions for all establishment and preservation obligations in Algorithm 1. Its fixed point $\mathsf{H}_{\mathcal{L}}(C)$ is jointly inductive, and every jointly inductive subset $C' \subseteq C$ satisfies $C' \subseteq \mathsf{H}_{\mathcal{L}}(C)$. Consequently, $I_{\mathsf{H}_{\mathcal{L}}(C)} \preceq I_{C'}$: the result is the strongest conjunction expressible as a subset of C.*

This is the classic Houdini greatest-fixpoint property (Flanagan & Leino, 2001), restated in our clause notation. In practice the per-obligation prover budget (Section 5.1) makes this oracle approximate.

RL scoring additionally removes clauses contradicted by observed reachable states $\mathcal{E}$ before Houdini:

$$\mathsf{PF}_{\mathcal{E}}(C) = \{c \in C : \forall p \in \mathcal{E},\ p \models c\}.$$

This positive filter is a cheap refutation step used only for RL scoring. Appendix C gives normalization and filtering details.

4.2 Inference Objective

At inference, the pipeline samples k masked responses and produces

$$C_k^{\text{pool}} = \bigcup_{i=1}^{k} \text{set}(\bar{A}_i), \qquad C_k = \mathsf{Filter}_{\mathcal{L}}(C_k^{\text{pool}}), \qquad I_k = I_{C_k}.$$

Only then is Q restored. The inference objective is the expected target-utility indicator,

$$\max_{\pi}\ \mathbb{E}_{A_{1:k}\sim\pi(\cdot|\mathcal{L})}\, \mathbf{1}[\text{Verify}(\mathcal{L}, I_k, Q)],$$

where Verify requires establishment, preservation, and every restored target at its original location; for one post-loop target, its last obligation is $I_k \wedge \neg B \Rightarrow q$. Its empirical counterpart is compose@k, which can succeed even when no standalone response is sufficient. The training sections below therefore optimize a target-hidden surrogate for this curve.

4.3 Target-Independent Strength Examples

To grade inductive sets without Q, we execute a deterministic, precondition-checked input sweep and record reachable loop-head states $\mathcal{E}$. From these executions we construct two negative-example families: local one- or two-variable perturbations that break relations, and continuations of the real body after a witnessed loop exit. Cleaning is deliberately conservative—any candidate that executions cannot rule out is dropped: it removes observed reachable states, possible fresh entries, duplicates, and candidates involving unsupported state. Unsupported or incomplete executions disable the affected family. We call this family-based construction the *structured* sampler; Section 5.6 contrasts it with uniform random perturbations. Figure 5, Algorithm 2, and exact budgets appear in Appendix C.1.

Let $\mathcal{N}$ be the retained negative traces. For a retained clause set C, $\text{rej}(C) \subseteq \mathcal{N}$ contains each trace on which at least one clause in C is false at some recorded state; singleton traces recover ordinary state rejection.

4.4 Distilling Multi-Response Search

SFT targets are built by the same rollout–decompose–pool–filter–compose loop that inference runs, driven by the untrained policy and accepted by the reward. For each masked loop $\mathcal{L}$ in the training corpus $\mathcal{D}_{\text{train}}$, round t samples a group of G_{sft}=16 target-hidden responses from the base model (Qwen3-8B), adds their clauses to the pool, and filters the union to an inductive set,

$$C_{\mathcal{L}}^{(t)} = \mathsf{Filter}_{\mathcal{L}}\Big(C_{\mathcal{L}}^{(t-1)} \cup \bigcup_{i=1}^{G_{\text{sft}}} \text{set}(\bar{A}_i^{(t)})\Big), \qquad C_{\mathcal{L}}^{(0)} = \varnothing.$$

The filtered candidate is scored on the negative traces of Section 4.3,

$$\text{cov}(C) = \frac{|\,\text{rej}(C)|}{|\mathcal{N}|}.$$

The search stops when $\mathrm{cov}(C_{\mathcal{L}}^{(t)}) \geq \tau$, with $C_{\mathcal{L}}^{\mathrm{sft}} = C_{\mathcal{L}}^{(t)}$; otherwise another group is sampled and added to the accumulated pool, for at most T_{sft}=5 rounds (80 responses). Loops that do not reach τ within this budget are dropped. Here coverage is an acceptance criterion for an already filtered, composed SFT target; Section 4.5 instead assigns clause-level credit within imperfect RL responses.

Before serialization, we remove normalized duplicates, tautologies, and clauses implied by other retained atomic clauses, while retaining all remaining clauses to preserve coverage. Every accepted set is serialized as one SFT target:

$$\mathcal{D}_{\mathrm{SFT}} = \{(\mathcal{L}, \mathrm{serialize}(C_{\mathcal{L}}^{\mathrm{sft}})) : \mathcal{L} \in \mathcal{D}_{\mathrm{train}},\ \mathrm{cov}(C_{\mathcal{L}}^{\mathrm{sft}}) \geq \tau\}.$$

Neither input nor target contains Q: SFT serializes a reward-accepted, Houdini-retained multi-response search result as one response.

4.5 Clause-Decomposed, Group-Compositional Reward

The reward design responds directly to the sharpening tendency described in Section 1. Group-relative optimization with a whole-response reward concentrates probability mass on a few high-reward responses, eroding the clause diversity that the pipeline composes at inference. We therefore decompose credit *within* each rollout—surviving clauses earn reward even when siblings fail—and re-allocate credit *across* the group by each rollout's marginal contribution to pooled coverage, paying the policy for expanding the joint clause pool rather than for reproducing its most common clauses.

For rollout i, first apply the clause-local positive filter, pool the group, and run the verifier filter once:

$$C_i^{\mathrm{pf}} = \mathsf{PF}_{\mathcal{E}}(\mathrm{set}(\bar{A}_i)), \qquad C_{\mathcal{G}}^{\mathrm{surv}} = \mathsf{Filter}_{\mathcal{L}}\left(\bigcup_{j=1}^{G} C_j^{\mathrm{pf}}\right).$$

We then use normalized semantic keys to assign each pooled survivor back to every rollout that proposed it,

$$C_i^{\mathrm{train}} = \{c \in C_i^{\mathrm{pf}} : \mathrm{key}(c) \in \mathrm{key}(C_{\mathcal{G}}^{\mathrm{surv}})\}, \qquad R_i = \mathrm{rej}(C_i^{\mathrm{train}}).$$

This projection implements *intra-rollout decomposition*: malformed or non-inductive clauses are removed without erasing useful clauses in the same response, while mutually supporting clauses from different responses are filtered under the same pooled context used at inference (Appendix L). When retained negatives exist, the reward adds no bonus merely for valid syntax or whole-response validity. The subset receives the dense, target-independent strength score

$$\mathrm{cov}(C_i^{\mathrm{train}}) = \frac{|R_i|}{|\mathcal{N}|}.$$

This and the following allocation apply when $\mathcal{N} \neq \varnothing$; when the sampler retains none, we use the binary fallback in Appendix C.2.

Independent coverage, however, rewards a complementary discovery exactly like one repeated by every peer. We therefore allocate credit for the union $\bigcup_i R_i$ across the rollout group $\mathcal{G} = \{1, \ldots, G\}$ as a coverage game. Let $f(\nu) = \sum_{j=1}^{G} \mathbf{1}[\nu \in R_j]$ count how many peers reject trace ν. The Shapley value of rollout i has the closed form (Shapley, 1953) (proof in Appendix B)

$$\phi_i = \frac{1}{|\mathcal{N}|} \sum_{\nu \in R_i} \frac{1}{f(\nu)}, \qquad \sum_{i=1}^{G} \phi_i = \frac{|\bigcup_i R_i|}{|\mathcal{N}|}. \tag{1}$$

Thus a unique rejection receives full credit, whereas $f(\nu)$ redundant rejections split it. The Shapley allocation is the unique credit rule satisfying efficiency, symmetry, linearity, and the null-player axiom (Shapley, 1953). It implements *inter-rollout composition*: the same response earns less group credit when its verified constraints are already supplied by peers.

The reward combines the two credits,

$$r_i = w_c\ \mathrm{cov}(C_i^{\mathrm{train}}) + w_g \phi_i, \tag{2}$$

where $(w_c, w_g) = (1.0, 0.3)$. We call the first term the *clause credit* and the second the *group (Shapley) credit*; Section 5.6 ablates each. GRPO normalizes these group-conditioned rewards within

the sampled response group. Each training step samples a group of $G{=}8$ responses per masked loop, scores them by Equation (2), and applies one GRPO update; this training-time group is distinct from the inference sweep over k (Section 4.2). Appendix C.2 gives the standard policy objective, clause normalization, and fallback behavior.

5 EXPERIMENTS

We ask how composition scales with sampling (RQ1), how much each pipeline component contributes (RQ2), how CRAFT compares with specialized tools (RQ3), how much RL adds (RQ4), and what drives the training gain (RQ5).

5.1 EXPERIMENTAL SETUP

Data and judge. Training data comprise 4,992 RL records and 33,346 SFT records (31,865 distinct program sources) of target-masked scalar-integer single-loop programs; SFT targets are synthesized with `gpt-5.6-luna` and verified by Frama-C/WP (Appendix G). Evaluation uses the standard benchmark suites of prior learning-based invariant-generation systems: 832 C programs, comprising 316 linear programs (133 Code2Inv (Si et al., 2020), 84 SyGuS 2019 (Alur et al., 2013), 99 SV-COMP 2024 (Beyer, 2023)) and 50 nonlinear-arithmetic (NLA) programs (30 LIPuS (Yu et al., 2023), 20 SV-COMP 2024) as released with Clause2Inv (Cao et al., 2025), plus 466 integer programs from Loopy (Kamath et al., 2023). After target removal, no training program matches an evaluation program under exact-text, alpha-renamed, or constant-abstracted matching. Appendix F reports the complete instance-overlap and structural-distribution audit. We remove non-contract comments and mask all target predicates before generation, then restore the untouched source only after invariants are fixed. Success requires Frama-C/WP (Kirchner et al., 2015), under a 5-second per-obligation prover budget, to prove establishment, preservation, and every restored target; all failures count as zero.

Inference protocol. All CRAFT evaluation runs share target masking, the canonical prompt, clause processing, and Houdini; external tools retain their own target-hidden orchestration. Within CRAFT, API models use `reasoning_effort=none`; all locally served checkpoints use non-thinking decoding to match data synthesis and training. Decoding uses temperature 1.0 and top-p 1.0. API calls cap completions at 8,192 tokens, including provider-internal reasoning tokens; local vLLM calls cap each response at 2,048 emitted tokens. We use no re-roll or target feedback. RQ1 reuses one saved pool of stochastic responses per task (32 for local checkpoints). Appendix H gives benchmark provenance, serving, and training details.

Metrics. Throughout evaluation, success means Q-sufficiency under the final verifier. pass@k is the probability that at least one of k responses proves Q. Whenever a saved pool contains c successful responses among its n responses, we use the unbiased estimator $\widehat{\text{pass@}}\,k = 1 - \binom{n-c}{k}/\binom{n}{k}$ for that program and average it over programs. compose@k is the fraction of programs proved by unioning the first k responses and applying Houdini. The two metrics capture different outcomes: pass@k measures standalone completion within k responses, whereas compose@k measures the deployed pipeline's filtered composition of those responses. Our prefix compose@k uses the first k saved responses rather than averaging over all $\binom{n}{k}$ subsets. We report k_{95}, the smallest budget in the measured grid $\{1, 4, 8, 16, 32\}$ whose compose@k reaches 95% of compose@32. A smaller k_{95} means the retained pool reaches its large-budget performance sooner.

5.2 RQ1: HOW DO PASS@K AND COMPOSE@K SCALE?

We sample 32 target-hidden responses per task from six base checkpoints: Qwen3-1.7B, Qwen3-4B, Qwen3-8B, Qwen3-14B, Qwen3-30B-A3B (Yang et al., 2025), and Llama 3.1 8B. Complete curves, k_{95}, and per-stratum breakdowns appear in Appendix 10.

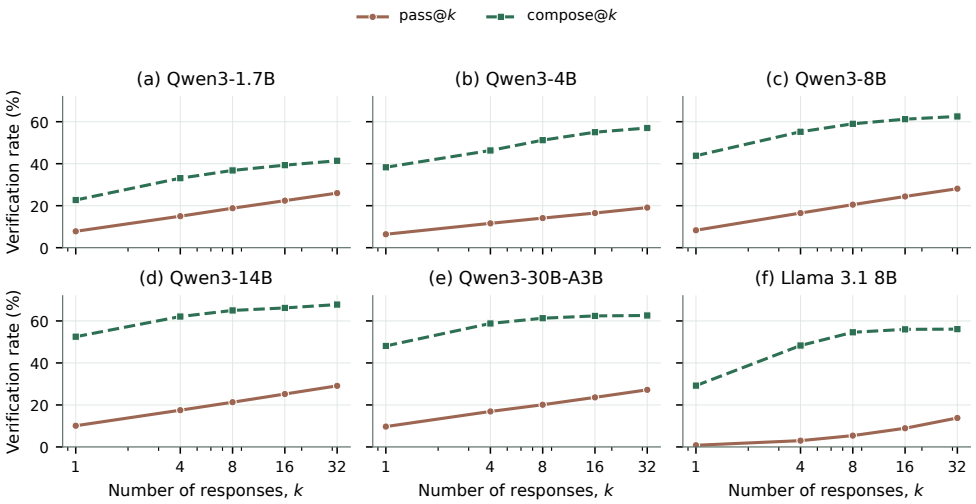

Figure 2: pass@k (unbiased estimate) and prefix compose@k. Composition saturates by k≈8–16, while pass@k keeps growing.

At k=1, compose@1 exceeds the unbiased pass@1 estimate by 14.9–42.4 points across checkpoints. This is an unpaired descriptive comparison: pass@1 is estimated from the full response pool, whereas compose@1 evaluates the saved first response. RQ2 measures the pipeline effect on matched responses. In the retained response pools, compose@16 is within 2.1 points of compose@32 on every checkpoint ($k_{95} \in \{8, 16, 32\}$; Appendix 10). In contrast, pass@8 reaches only 72–74% of pass@32 on the five Qwen checkpoints and gains another 5.0–7.8 points by $k = 32$. Llama 3.1 8B is harder in standalone proof generation—pass@32 is 13.8%—but shows the same compositional pattern: compose rises from 29.2% at $k = 1$ to 56.0% at $k = 16$, then changes by only 0.1 points. Composition thus finds most of its useful clauses within the first dozen samples, whereas standalone proofs require wider sampling. The compose@32 values (41.4–67.8%) further suggest that these pools remain limited by clause quality or diversity: extending the budget to 32 responses does not remove the gap.

Insight. Standalone completion and filtered composition scale differently with additional sampling. Across the six checkpoints, compose@16 is within 2.1 points of compose@32, while pass@k continues to improve through k=32. Reporting both metrics separates the quality of individual responses from the utility of their composed clause pool.

5.3 RQ2: What Does Each Component Contribute?

The main neural experiment evaluates saved responses per program at budgets k=1 and k=8 and decomposes the pipeline into cumulative stages. Framework-free rows judge each response as one indivisible conjunction (pass@k); pipeline rows decompose the same responses into clauses, filter them, and compose the survivors (compose@k). Zero denotes the untrained checkpoint. The three Qwen3 models add the training stages (+SFT, +SFT+RL) and two ablation rows that repeat the trained checkpoints without the pipeline; Llama 3.1 8B reports the SFT rows without RL.

	Linear / 316		NLA / 50		Loopy / 466		All / 832	
Stage	k=1	k=8	k=1	k=8	k=1	k=8	k=1	k=8
Qwen3-4B								
Zero, no pipeline (pass)	6.2	14.1	0.5	3.0	7.2	15.2	6.4	14.1
Zero + pipeline (compose)	38.6	51.9	8.0	8.0	41.4	55.4	38.3	51.2
+SFT	TBD	TBD	TBD	TBD	TBD	TBD	TBD	TBD
+SFT+RL	TBD	TBD	TBD	TBD	TBD	TBD	TBD	TBD
SFT, no pipeline (pass)	TBD	TBD	TBD	TBD	TBD	TBD	TBD	TBD
SFT+RL, no pipeline (pass)	TBD	TBD	TBD	TBD	TBD	TBD	TBD	TBD
Qwen3-8B								
Zero, no pipeline (pass)	8.6	20.7	3.2	5.9	8.6	22.0	8.3	20.5
Zero + pipeline (compose)	43.7	60.4	6.0	12.0	47.9	63.1	43.8	59.0
+SFT	TBD	TBD	TBD	TBD	TBD	TBD	TBD	TBD
+SFT+RL	TBD	TBD	TBD	TBD	TBD	TBD	TBD	TBD
SFT, no pipeline (pass)	TBD	TBD	TBD	TBD	TBD	TBD	TBD	TBD
SFT+RL, no pipeline (pass)	TBD	TBD	TBD	TBD	TBD	TBD	TBD	TBD
Qwen3-14B								
Zero, no pipeline (pass)	8.9	19.3	1.1	4.9	11.8	24.3	10.1	21.3
Zero + pipeline (compose)	54.1	68.0	8.0	12.0	56.2	68.7	52.5	65.0
+SFT	TBD	TBD	TBD	TBD	TBD	TBD	TBD	TBD
+SFT+RL	TBD	TBD	TBD	TBD	TBD	TBD	TBD	TBD
SFT, no pipeline (pass)	TBD	TBD	TBD	TBD	TBD	TBD	TBD	TBD
SFT+RL, no pipeline (pass)	TBD	TBD	TBD	TBD	TBD	TBD	TBD	TBD
Llama 3.1 8B								
Zero + pipeline (compose)	29.4	55.4	2.0	14.0	32.0	58.4	29.2	54.6
+SFT	TBD	TBD	TBD	TBD	TBD	TBD	TBD	TBD
SFT, no pipeline (pass)	TBD	TBD	TBD	TBD	TBD	TBD	TBD	TBD

Table 1: Cumulative contribution of each pipeline component, by benchmark stratum (% verified, `reasoning_effort=none`/non-thinking). Framework-free rows report pass@k; pipeline rows report compose@k on the same saved responses, so adjacent rows isolate one component. RL is trained on the three Qwen3 models; Llama 3.1 8B reports SFT rows only. Complete probe curves and per-stratum results for all six models appear in Appendix 10. API models under the same protocol appear in Table 17 (Appendix I.6).

Because framework-free and pipeline rows reuse the same saved responses, their paired difference isolates the net effect of clause extraction, normalization, and Houdini without additional model calls: composing lifts every model by 28.4–42.4 points at k=1 and by 37.1–49.2 points at k=8 in aggregate. All rows report full-workload means over the same 832 programs. Per-model probe curves appear in Appendix 10.

5.4 RQ3: How Does CRAFT Compare with Specialized Tools?

We compare CRAFT with the target-hidden pipelines of AutoSpec, SESpec, Clause2Inv, Loopy, and Daikon (Wen et al., 2024; Yang et al., 2026b; Cao et al., 2025; Kamath et al., 2023; Ernst et al., 2007). All LLM-based systems use `gpt-5-nano`, and all outputs are judged on the same untouched programs by Frama-C/WP. Each system retains its native, predeclared orchestration, so model-call and token budgets differ; Table 2 reports both token and runtime costs. Unsupported inputs count as failures in the common 832-program denominator.

The Naive baseline uses one call without invariant-specific instructions. In contrast, CRAFT prompts for a diverse clause pool and filters the result before composition. Consequently, its raw responses have lower standalone validity than Naive (pass@1: 3.61% vs. 16.47%), while its filtered composition reaches 33.89% at the same one-response budget.

System	Linear / 316	NLA / 50	Loopy / 466	All / 832	Tokens / task	Time / task
AutoSpec	47.78%	6.00%	42.27%	42.19%	2,181.72	27.34 s
SESpec	28.80%	4.00%	33.05%	29.69%	22,081.61	68.50 s
Clause2Inv	20.89%	0.00%	0.00%	7.93%	1,056.25	8.41 s
Daikon	23.10%	0.00%	21.67%	20.91%	0	4.89 s
Loopy	20.25%	0.00%	19.74%	18.75%	14,513.37	147.16 s
Naive	15.19%	2.00%	18.88%	16.47%	225.95	2.98 s
CRAFT (pass@1)	2.53%	0.00%	4.72%	3.61%	1,135.83	7.86 s
CRAFT (compose@1)	34.81%	22.00%	34.55%	33.89%	1,136.82	24.52 s
CRAFT (compose@4)	48.10%	30.00%	51.72%	49.04%	1,905.10	46.40 s
CRAFT (compose@8)	55.38%	44.00%	56.87%	55.53%	2,929.47	69.99 s

Table 2: Common target-hidden comparison with `reasoning_effort=none`, using the same untouched inputs and Frama-C/WP final judge. All LLM-driven rows share the same fixed backbone model; tokens and time are mean totals per task. The three compose rows are recomputed from one archived ten-rollout pool by prefix-subset composition, unioning the first k responses through the same filter chain; their tokens count one prompt plus k archived completions, and their time adds the measured filter-and-judge time at budget k to the single-request latency.

The archived reasoning-enabled comparison appears in Appendix 18. On three further API models the same pairing lifts compose@1 over pass@1 by 18.7–23.4 points (mean 20.9; Appendix 17), compared with the backbone's 30.3: the gain appears across models and compresses as single-shot accuracy rises. Under this setting, increasing CRAFT from four to eight candidates raises verification from 49.0% to 55.5%.

Appendix I.8 documents system-specific interface constraints under target hiding. The complete per-tool results and the archived medium-reasoning comparison also appear there.

Finding. Under a common target-hidden evaluation and each system's native fixed orchestration, CRAFT verifies 49.0% of the 832 programs with four composed responses, compared with 42.2% for the strongest external baseline. This is an end-to-end system comparison; the systems differ in orchestration and computational budget.

5.5 RQ4: How Much Does Reinforcement Learning Improve Inference?

We evaluate matched Qwen3-8B before–after RL paths from the Zero and SFT initializations: Zero is the untrained checkpoint and RL-Zero its RL result; SFT and SFT+RL are the corresponding SFT-path checkpoints. Inference is fixed within each pair; RQ1 is not used to estimate the RL effect.

What to expect. If RL with whole-response credit merely sharpens the distribution (Yue et al., 2025), it should raise pass@1 while leaving or degrading large-k coverage. Clause-level credit with group allocation predicts the opposite pattern: pass@k stays near the initialization, while compose@k shifts up at small budgets without degrading at large ones.

The SFT stage distills the filtered multi-response union into single responses (Section 4.4); matched SFT runs on the curated pool are TBD (Table 3), with full curves across four backbones in Appendix I.2.

Checkpoint	Stage	pass@1	compose@1	compose@8	compose@32	k_{95}
Zero	before RL	8.3%	43.8%	59.0%	62.5%	16
RL-Zero	after RL	6.80%	57.93%	71.27%	74.28%	8
SFT	before RL	TBD	TBD	TBD	TBD	TBD
SFT+RL	after RL	TBD	TBD	TBD	TBD	TBD

Table 3: Matched Qwen3-8B evaluation before and after RL.

`rl_panels.pdf` placeholder — generated by `figures/plot_rl_panels.py` from the canonical RL pool artifact (pass@k left, compose@k right with the SFT compose@32 reference line and the crossing budget k^* annotated).

Figure 3: Response-level vs. compositional effects of RL (Qwen3-8B, SFT initialization). (a) pass@k is expected to remain flat, consistent with redistribution rather than support expansion (Yue et al., 2025). (b) compose@k shifts upward at small budgets: the RL policy reaches its initialization's compose@32 (dotted line) at k^* samples, and compose@32 itself does not degrade.

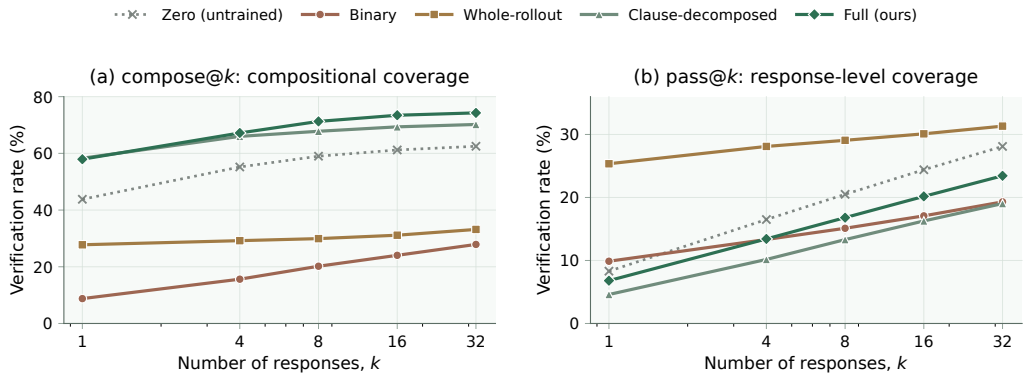

Figure 4: Reward-function ablation on Qwen3-8B (Zero initialization). Whole-rollout credit raises pass@k (b) but leaves compose@k (a) below the untrained model; binary credit collapses both. Clause-decomposed credit reverses the trade-off, and Full is best at every budget from k=4. Dotted: Zero, the untrained checkpoint. Exact numbers in Table 15 (Appendix I.4).

Results. Consistent with the redistribution view, response-level accuracy does not improve after RL (Figure 3, left; pass@1 TBD vs. TBD). The redistribution is instead steered toward the complementary tail of the clause pool: compose@k^* with k^*=TBD matches its initialization's compose@32 (TBD), an effective TBD-fold budget reduction, and compose@32 itself does not degrade (TBD vs. TBD). RL thus packs into k^* samples the useful clauses its initialization produces only across thirty-two. A clause-frequency analysis against TBD initialization samples attributes part of the gain to clauses the initialization produces with probability below TBD per response.

5.6 RQ5: What Drives the Training Gain?

We run two ablations, one over the reward function and one over the negative sampler. Both train Qwen3-8B from the Zero initialization on the curated pool with all other training and inference settings fixed; runs within an ablation differ only in the reward or the sampler. Appendix I.4 gives the exact reward definitions and complete numbers.

Reward-function ablation. Four variants form a ladder. Binary pays $\{0, 1\}$ for a fully inductive response; Whole-rollout pays the strength score $\mathrm{cov}(C_i^{\mathrm{train}})$ only when every clause survives; Clause-decomposed pays it for the surviving subset C_i^{train} alone; Full adds the group (Shapley) credit $w_g\phi_i$ of Equation (2). The first two are response-level credit, the last two clause-level; all share the clause cap K=20.

Figure 4 reveals three effects. *First, response-level credit collapses composition.* Whole-rollout credit sharpens the policy toward a few already-supported answers: pass@1 rises by 17 points, but the clause pool loses diversity, and composing more responses adds little (compose@32 33.2%). Binary credit collapses both metrics. Both response-level variants stay 16–40 points below the untrained model's compose curve at every budget—the sharpening anticipated in Section 5.5. *Second, credit must be decomposed to clauses.* Once each surviving clause is rewarded for the negatives it alone rejects, the policy spreads mass over complementary clauses: compose@k is 2.1–2.3 times the whole-rollout variant at every budget and 2.5–6.6 times the binary variant, and the compose curve keeps rising through k=32. *Third, group credit raises the ceiling.* Adding Shapley credit on top of clause-decomposed strength improves composition at every budget beyond one response (+1.2 at k=4, +3.5 at k=8, +4.1 at k=16 and 32, reaching 74.3%) and adds 2.2–4.4 points of pass@k at every budget, while trailing by 0.4 points at compose@1: Full is the best variant from k=4 on. Its pass@k nevertheless remains 1.5–4.7 points below the untrained model; the training gain is compositional.

Why credit structure decides the outcome. Whole-rollout credit grades each response as an indivisible unit: one failed clause erases the credit of every other, so the safest policy is to repeat the few responses that already pass in full—the sharpening that raises pass@1 while shrinking the clause pool (Yue et al., 2025). Clause-level credit removes this gate. A partially correct response is still paid for what survives, so the policy keeps proposing clauses whose union proves more than any single response; this is the larger effect (+7.7 to +14.5 compose points over the untrained model). Group credit then grades each rollout against its peers: a clause every rollout repeats earns less than a complementary one, so the objective becomes the group's union rather than any single response. This adds a further 1.2–4.1 points from k=4 and, consistent with paying for coherent rather than merely numerous clauses, restores 2.2–4.4 points of pass@k. What RL does to the sampling distribution is decided by what the reward values.

Finding (credit structure). Response-level credit sharpens rather than expands the sampled support: whole-rollout credit raises pass@1 by 17 points, yet both response-level variants stay 16–40 points below the untrained model on compose@k at every budget, and binary credit also falls below it on pass@k from k=4. Decomposing credit to clauses reverses this: compose@k exceeds the untrained model at every budget and at least doubles either response-level variant. Group (Shapley) credit then lifts the curve further from k=4 on (compose@32 74.3% vs. 70.2%) and adds 2.2–4.4 points of pass@k.

Negative-sampler ablation. Under the full reward, the structured sampler (relation and post-exit families, Section 4.3) exceeds uniform random perturbations by 5.5–6.7 compose@k points at every budget, whereas pass@k differs by -1.8 to $+1.6$ points (Table 16, Appendix I.4).

Does negative coverage predict hidden-target utility? We score 6,190 saved target-hidden candidate sets from eight generation pipelines on the structured negatives, then use the restored-target Frama-C/WP verdict only as an evaluation label. Among the 5,606 candidates with a continuous coverage score, coverage predicts target success with AUROC 0.738 (program-clustered 95% CI 0.711–0.763) and AUPRC 0.730, against a 0.508 positive-rate baseline. More stringently, ranking candidates only against other candidates for the same program gives macro AUROC 0.794 (0.773–0.814); a within-program permutation test gives $p < 2 \times 10^{-4}$. The association is strong on Linear and Loopy but weak on NLA, exposing a limitation of the current negative construction rather than establishing coverage as a universal proxy. Appendix I.5 gives the protocol, strata, and diagnostic curves.

6 Conclusion and Limitations

CRAFT makes filtered clause composition the inference target: its pipeline composes rollouts, SFT distills their union, and target-independent RL rewards useful clauses and complementary coverage.

The recipe transfers beyond loop invariants. It applies to problems whose answers (i) decompose into independently checkable units composed by conjunction, (ii) admit a cheap mechanical filter for unit validity, (iii) include valid-but-weak degenerate solutions that pass/fail cannot grade, and (iv) allow

behavioral negatives, on which weak and strong solutions disagree, to be derived from executions or simulations. Test generation judged by mutant killing, auxiliary-lemma synthesis for proof assistants, and data-constraint discovery all satisfy these conditions.

Evidence is limited to scalar-integer, single-loop C programs, Frama-C/WP and ACSL, and vLLM-served local checkpoints. The provenance allocation conditions credit on the sampled full group and is not an exponential coalition-value Shapley computation. Appendices J and L detail these scope and reward approximations.

AI Use Statement

Generative AI tools are integral to the evaluated system: they produce the invariant candidates and synthetic SFT data described in the paper. They were also used during research to refine hypotheses, methodology, and experiments; formulate and check mathematical claims; implement and test software; analyze and interpret experimental artifacts; prepare figures and tables; search and summarize literature; and draft, edit, and translate manuscript text. The authors reviewed all AI-assisted work and tested generated code. They take responsibility for independently validating the final content, empirical claims, and released artifacts.

Reproducibility Statement

The method and objective are specified in Section 4; Appendices C–H give the implementation defaults, prompt, training interface, data-overlap audit, and evaluation protocol. Appendix I organizes the available measurements, including per-seed results.

References

Rajeev Alur, Rastislav Bodík, Garvit Juniwal, Milo M. K. Martin, Mukund Raghothaman, Sanjit A. Seshia, Rishabh Singh, Armando Solar-Lezama, Emina Torlak, and Abhishek Udupa. Syntax-guided synthesis. In *Proc. Formal Methods in Computer-Aided Design (FMCAD)*, pp. 1–8, 2013.

Patrick Baudin, Pascal Cuoq, Jean-Christophe Filliâtre, Claude Marché, Benjamin Monate, Yannick Moy, and Virgile Prevosto. ACSL: ANSI/ISO C specification language. `https://frama-c.com/html/acsl.html`, 2021.

Dirk Beyer. Competition on software verification (SV-COMP). `https://sv-comp.sosy-lab.org/`, 2023.

Weining Cao, Guangyuan Wu, Tangzhi Xu, Yuan Yao, Hengfeng Wei, Taolue Chen, and Xiaoxing Ma. Clause2Inv: A generate-combine-check framework for loop invariant inference. *Proceedings of the ACM on Software Engineering*, 2(ISSTA):1009–1030, 2025. doi: 10.1145/3728920.

Saikat Chakraborty, Shuvendu K. Lahiri, Sarah Fakhoury, Madanlal Musuvathi, Akash Lal, Aseem Rastogi, Aditya Senthilnathan, Rahul Sharma, and Nikhil Swamy. Ranking LLM-generated loop invariants for program verification. In *Findings of the Association for Computational Linguistics: EMNLP*, pp. 9164–9175, 2023. doi: 10.18653/v1/2023.findings-emnlp.614.

Zhipeng Chen, Xiaobo Qin, Youbin Wu, Yue Ling, Qinghao Ye, Wayne Xin Zhao, and Guang Shi. Pass@k training for adaptively balancing exploration and exploitation of large reasoning models. *arXiv preprint arXiv:2508.10751*, 2025.

Michael A. Colón, Sriram Sankaranarayanan, and Henny B. Sipma. Linear invariant generation using non-linear constraint solving. In *Proc. Int. Conf. on Computer Aided Verification (CAV)*, pp. 420–432. Springer, 2003.

Patrick Cousot and Radhia Cousot. Abstract interpretation: A unified lattice model for static analysis of programs by construction or approximation of fixpoints. In *Proc. 4th ACM SIGACT-SIGPLAN Symp. on Principles of Programming Languages (POPL)*, pp. 238–252, 1977.

Michael D. Ernst, Jeff H. Perkins, Philip J. Guo, Stephen McCamant, Carlos Pacheco, Matthew S. Tschantz, and Chen Xiao. The Daikon system for dynamic detection of likely invariants. *Science of Computer Programming*, 69:35–45, 2007.

Emily First, Markus N. Rabe, Talia Ringer, and Yuriy Brun. Baldur: Whole-proof generation and repair with large language models. In *Proc. 31st ACM Joint European Software Engineering Conf. and Symp. on the Foundations of Software Engineering (ESEC/FSE)*, pp. 1229–1241, 2023.

Cormac Flanagan and K. Rustan M. Leino. Houdini, an annotation assistant for ESC/Java. In *Proc. Int. Symp. of Formal Methods Europe (FME)*, pp. 500–517. Springer, 2001.

Robert W. Floyd. Assigning meanings to programs. In *Proc. Symp. in Applied Mathematics: Mathematical Aspects of Computer Science*, volume 19, pp. 19–32, 1967.

Pranav Garg, Christof Löding, P. Madhusudan, and Daniel Neider. ICE: A robust framework for learning invariants. In *Proc. Int. Conf. on Computer Aided Verification (CAV)*, pp. 69–87. Springer, 2014.

Andre He, Daniel Fried, and Sean Welleck. Rewarding the unlikely: Lifting GRPO beyond distribution sharpening. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, 2025.

C. A. R. Hoare. An axiomatic basis for computer programming. *Communications of the ACM*, 12 (10):576–580, 1969.

Zhechong Huang, Zhao Zhang, Zeyu Sun, Huifeng Sun, and Yingfei Xiong. Reinforcement learning with negative tests as completeness signal for formal specification synthesis. *arXiv preprint arXiv:2604.05820*, 2026.

Adharsh Kamath, Aditya Senthilnathan, Saikat Chakraborty, Pantazis Deligiannis, Shuvendu K. Lahiri, Akash Lal, Aseem Rastogi, Subhajit Roy, and Rahul Sharma. Finding inductive loop invariants using large language models. *arXiv preprint arXiv:2311.07948*, 2023.

Michael Karr. Affine relationships among variables of a program. *Acta Informatica*, 6(2):133–151, 1976.

Zachary Kincaid, Jason Breck, Ashkan Forouhi Boroujeni, and Thomas Reps. Compositional recurrence analysis revisited. In *Proc. ACM SIGPLAN Conf. on Programming Language Design and Implementation (PLDI)*, pp. 248–262, 2017.

Florent Kirchner, Nikolai Kosmatov, Virgile Prevosto, Julien Signoles, and Boris Yakobowski. Frama-c: A software analysis perspective. *Formal Aspects of Computing*, 27(3):573–609, 2015.

Hung Le, Yue Wang, Akhilesh Deepak Gotmare, Silvio Savarese, and Steven C. H. Hoi. CodeRL: Mastering code generation through pretrained models and deep reinforcement learning. In *Proc. Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

Chang Liu, Xiwei Wu, Yuan Feng, Qinxiang Cao, and Junchi Yan. Towards general loop invariant generation: A benchmark of programs with memory manipulation. In *Proc. Advances in Neural Information Processing Systems (NeurIPS)*, 2024a.

Ruibang Liu, Minyu Chen, Ling-I Wu, Jingyu Ke, and Guoqiang Li. Enhancing automated loop invariant generation for complex programs with large language models. *arXiv preprint arXiv:2412.10483*, 2024b.

Lezhi Ma, Shangqing Liu, Yi Li, Xiaofei Xie, and Lei Bu. SpecGen: Automated generation of formal program specifications via large language models. In *Proc. Int. Conf. on Software Engineering (ICSE)*, 2025.

Kenneth L. McMillan. Interpolation and SAT-based model checking. In *Proc. Int. Conf. on Computer Aided Verification (CAV)*, pp. 1–13. Springer, 2003.

Ido Pinto, Yizhak Yisrael Elboher, Haoze Wu, Nina Narodytska, and Guy Katz. Not all invariants are equal: Curating training data to accelerate program verification with SLMs. *arXiv preprint arXiv:2603.15510*, 2026.

Gabriel Ryan, Justin Wong, Jianan Yao, Ronghui Gu, and Suman Jana. CLN2INV: Learning loop invariants with continuous logic networks. In *Proc. Int. Conf. on Learning Representations (ICLR)*, 2020.

Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. DeepSeekMath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.

Lloyd S. Shapley. A value for n-person games. In Harold W. Kuhn and Albert W. Tucker (eds.), *Contributions to the Theory of Games II*, volume 28 of *Annals of Mathematics Studies*, pp. 307–317. Princeton University Press, 1953.

Xujie Si, Hanjun Dai, Mukund Raghothaman, Mayur Naik, and Le Song. Learning loop invariants for program verification. In *Proc. Advances in Neural Information Processing Systems (NeurIPS)*, pp. 7751–7762, 2018.

Xujie Si, Aaditya Naik, Hanjun Dai, Mayur Naik, and Le Song. Code2Inv: A deep learning framework for program verification. In *Proc. Int. Conf. on Computer Aided Verification (CAV)*, pp. 151–164. Springer, 2020.

Christian Walder and Deep Karkhanis. Pass@k policy optimization: Solving harder reinforcement learning problems. In *Advances in Neural Information Processing Systems*, volume 38, 2025.

Anjiang Wei, Tarun Suresh, Tianran Sun, Haoze Wu, Ke Wang, and Alex Aiken. InvBench: Can LLMs accelerate program verification with invariant synthesis? *arXiv preprint arXiv:2509.21629*, 2025.

Cheng Wen, Jialun Cao, Jie Su, Zhiwu Xu, Shengchao Qin, Mengda He, Haokun Li, Shing-Chi Cheung, and Cong Tian. Enchanting program specification synthesis by large language models using static analysis and program verification. In *Proc. Int. Conf. on Computer Aided Verification (CAV)*, pp. 302–328. Springer, 2024.

Guangyuan Wu, Weining Cao, Yuan Yao, Hengfeng Wei, Taolue Chen, and Xiaoxing Ma. LLM meets bounded model checking: Neuro-symbolic loop invariant inference. In *Proc. IEEE/ACM Int. Conf. on Automated Software Engineering (ASE)*, 2024a.

Haoze Wu, Clark Barrett, and Nina Narodytska. Lemur: Integrating large language models in automated program verification. In *Proc. Int. Conf. on Learning Representations (ICLR)*, 2024b.

Chuanhao Yan, Fengdi Che, Xuhan Huang, Xu Xu, Xin Li, Yizhi Li, Xingwei Qu, Jingzhe Shi, Chenghua Lin, Yaodong Yang, Binhang Yuan, Hang Zhao, Yu Qiao, Bowen Zhou, and Jie Fu. Re:Form—reducing human priors in scalable formal software verification with RL in LLMs: A preliminary study on Dafny. *arXiv preprint arXiv:2507.16331*, 2025.

An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, et al. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.

Fanpeng Yang, Xing Li, Shuling Wang, Jie An, Zeyu Sun, Shenghua Feng, Wenhan Wang, Weiyi Wang, Naijun Zhan, and Fanjiang Xu. How powerful are LLMs in generating formal program specifications? In *Proc. Int. Conf. on Machine Learning (ICML)*, 2026a.

Fanpeng Yang, Xu Ma, Shuling Wang, Xiong Xu, Qinxiang Cao, Naijun Zhan, Xiaofeng Li, and Bin Gu. Integrating symbolic execution with LLMs for automated generation of program specifications. *arXiv preprint arXiv:2506.09550*, 2026b.

Jianan Yao, Gabriel Ryan, Justin Wong, Suman Jana, and Ronghui Gu. Learning nonlinear loop invariants with gated continuous logic networks. In *Proc. ACM SIGPLAN Conf. on Programming Language Design and Implementation (PLDI)*, pp. 106–120, 2020.

Shiwen Yu, Ting Wang, and Ji Wang. Loop invariant inference through SMT solving enhanced reinforcement learning. In *Proc. ACM SIGSOFT Int. Symp. on Software Testing and Analysis (ISSTA)*, pp. 175–187, 2023. doi: 10.1145/3597926.3598047.

Yang Yue, Zhiqi Chen, Rui Lu, Andrew Zhao, Zhaokai Wang, Yang Yue, Shiji Song, and Gao Huang. Does reinforcement learning really incentivize reasoning capacity in LLMs beyond the base model? In *Advances in Neural Information Processing Systems*, volume 38, 2025.

A Scope and Motivating Example

We focus on numerical loops because their arithmetic equalities and inequalities, including polynomial relations, admit scalable automatic checking. This controlled setting factors out arrays, heaps, recursive data structures, auxiliary functions, and inductive predicates, allowing the study to isolate algebraic discovery and loop dynamics. Concretely, we consider one braced `while` loop over scalar C integers, including nonlinear updates. Even this restricted setting can hide nonlinear relations across coupled recurrences. For example:

```
void f(int k, int z) {
  int x = 1, y = 1, c = 1;
  while (c < k) { c++; x = x*z + 1; y = y*z; }
  /*@ assert 1+x*z-x-z*y == 0; */
}
```

Case study: target visibility turns discovery into copying. If the assertion is visible to the generator, the task is nearly trivial: the assertion itself can be copied verbatim as a loop invariant. It holds initially because $x = y = 1$. Writing $I(x, y) = 1 + xz - x - zy$, one loop iteration gives

$$I(xz + 1, yz) = 1 + (xz + 1)z - (xz + 1) - z(yz) = zI(x, y),$$

so $I = 0$ is inductive and directly discharges the exit assertion. Under the target-hidden protocol, however, the model sees the initialization and the two coupled recurrences but not the assertion, and must independently recover this relation. The archived non-reasoning rollout pools do so in three algebraically equivalent forms:

Model	First verified prefix	Rediscovered invariant clause
GPT-5	compose@1	$x - 1 = z(x - y)$
DeepSeek V4-Flash	compose@4	$x(z - 1) = zy - 1$
Claude Sonnet-4.6	compose@8	$x(z - 1) = yz - 1$

All three clauses rearrange to the hidden assertion. None of the ten raw responses in any of these three pools passes as a whole on this program; the inference filter removes incompatible siblings and retains the useful relation. Thus target hiding prevents a copy-the-goal shortcut, while the successful compositions demonstrate recovery from loop dynamics rather than access to the postcondition.

B Proofs of Compositionality

This section contains all proofs for the formal composition claims in Section 4.1. Write $\mathcal{L} = (P, B, S)$, where P denotes loop-entry states, B the guard, and S the body transition.

B.1 Conjunction of Valid Invariants

Proof of Proposition 1. For each j, validity gives $P \Rightarrow I_j$; hence $P \Rightarrow \bigwedge_j I_j = I$. Suppose $I \wedge B$ holds before one execution of S. Then $I_j \wedge B$ holds for every j, so preservation of I_j establishes every I_j afterward. Therefore I is preserved and is a valid invariant.

Because a conjunction implies each conjunct, $I \preceq I_j$. It satisfies $I \prec I_j$ precisely when the reverse implication fails. Since

$$I_j \Rightarrow I \quad \Longleftrightarrow \quad I_j \Rightarrow \bigwedge_{i \neq j} I_i,$$

the stated condition follows. In particular, two valid invariants form a strict strengthening of both exactly when neither invariant implies the other; without this non-redundancy condition, conjunction is only guaranteed to be no weaker. □

B.2 Mutual Support Among Arbitrarily Many Clauses

Proof of Proposition 2. The first premise is exactly establishment of I_C. For preservation, assume $I_C \wedge B$ before executing S. The second premise establishes each c_j afterward. Since every conjunct then holds in the same successor state, their conjunction I_C also holds. Thus $\{I_C \wedge B\}S\{I_C\}$, and I_C is valid.

No standalone preservation premise appears in this argument. A clause may use all other conjuncts in the pre-state, so clauses that fail preservation separately may still be preserved jointly. Establishment is different: $P \Rightarrow I_C$ implies $P \Rightarrow c_j$ for every j. Hence a candidate false at some loop-entry state cannot be rescued by conjunction. □

The phenomenon is not limited to pairs. For any $m \geq 2$, consider the cyclic transition, where x_i' denotes the value of x_i after one execution of S,

$$x_1' = x_2, \quad x_2' = x_3, \quad \ldots, \quad x_{m-1}' = x_m, \quad x_m' = x_1$$

with entry condition $P \equiv \bigwedge_i x_i = 0$, and candidates $c_i \equiv x_i \geq 0$. Each c_i is established. It is not preserved alone: a state can satisfy $x_i \geq 0$ while its predecessor under the cyclic assignment is negative. Nevertheless, the conjunction $\bigwedge_i x_i \geq 0$ is preserved because every successor coordinate is an old coordinate constrained by another conjunct. Thus all m candidates can fail standalone preservation while forming one valid, mutually supported invariant.

B.3 Greatest Jointly Inductive Candidate Subset

Let $C_0 = C$, and let $C_{j+1} \subseteq C_j$ be the set remaining after one ideal Houdini deletion round in Algorithm 1. Because C is finite, this descending sequence reaches the fixed point $\mathsf{H}_{\mathcal{L}}(C)$.

Proof of Theorem 3. At the fixed point, every $c \in \mathsf{H}_{\mathcal{L}}(C)$ satisfies establishment and preservation under the full conjunction $I_{\mathsf{H}_{\mathcal{L}}(C)}$. Applying Proposition 2 shows that this conjunction is valid.

It remains to prove greatestness. Let $C' \subseteq C$ be any jointly inductive subset. We show by induction over deletion rounds that $C' \subseteq C_j$. The base case $C' \subseteq C_0 = C$ is immediate. Assume $C' \subseteq C_j$, and take any $c \in C'$. Joint establishment of C' gives $P \Rightarrow c$, so c cannot be deleted for establishment. Joint preservation gives

$$\{I_{C'} \wedge B\}\, S\, \{c\}.$$

Since $C' \subseteq C_j$, the antecedent $I_{C_j} \wedge B$ implies $I_{C'} \wedge B$. Strengthening a valid Hoare precondition preserves validity, so

$$\{I_{C_j} \wedge B\}\, S\, \{c\}$$

also holds. Therefore no member of C' is deleted in round j, proving $C' \subseteq C_{j+1}$. At termination, $C' \subseteq \mathsf{H}_{\mathcal{L}}(C)$. Conjunction reverses set inclusion, hence $I_{\mathsf{H}_{\mathcal{L}}(C)} \preceq I_{C'}$. □

The theorem is deliberately candidate-relative: Houdini cannot synthesize a missing clause, so it need not recover the strongest invariant expressible in the full assertion language. It is also an ideal-oracle result. The inference-time Frama-C/WP backend may time out or fail to prove a valid obligation; such clauses are conservatively removed. The final retained conjunction remains accepted by the verifier, but completeness and greatestness are not claimed for these resource-bounded executions.

B.4 Closed Form of the Coverage-Game Shapley Value

The allocation of Equation (1) is the Shapley value of the coverage game $v(\mathcal{S}) = |\bigcup_{i\in\mathcal{S}} R_i|/|\mathcal{N}|$, for coalitions $\mathcal{S} \subseteq \mathcal{G}$ of the rollout group.

Proposition 4 (Closed form of the coverage game)**.** *For the coverage game $v(\mathcal{S}) = |\bigcup_{i\in\mathcal{S}} R_i|/|\mathcal{N}|$, the Shapley value is $\phi_i = \frac{1}{|\mathcal{N}|}\sum_{\nu\in R_i}\frac{1}{f(\nu)}$, and $\sum_i \phi_i = v(\mathcal{G})$.*

Constant	Value	Role
runs requested	12	input executions; deterministic no-input programs collapse to one, and oracle-body sampling doubles the request
sampler seed	0	one canonical example set per reward request unless overridden
input tiers	20 fixed values	deterministic near-input schedule listed below; unconstrained tier candidates are clamped to $[-64, 64]$
far probes	3, plus 2 ordered multi-parameter probes	seed-hashed input-envelope coverage, magnitude ≤ 512
relation deltas	dense: $\pm\{1, 2\}$; terminal: $\pm\{1, 2, 3, 5, 8\}$	single-axis and pairwise perturbations; terminal bases require a witnessed final transition rather than a forward dense window
dense window	3	sampled successors required for a relation base
relation base cap	96	bases stratified across positives
post-exit steps	24	continuation length after a genuine exit (one negative trace per run)
negative-example budgets	48 / 12	relational / post-exit traces (sampler schema v7); stable round-robin across structural buckets, nearest perturbations first
iteration cap	2×10^6	divergence safety net (capped runs flagged)
(w_c, w_g)	$(1.0, 0.3)$	negative rejection / Shapley group-credit weights
K	20 clauses	response cap applied independently to every generated response; later clauses are discarded
G_{sft}	16 responses	group sampled per SFT strengthening round (§4.4)
T_{sft}	5 rounds	maximum strengthening rounds per loop (at most 80 responses)
τ	TBD	SFT acceptance threshold on the composed set's negative coverage (whole-rollout strength)
Houdini iterations	≤ 500	greatest-inductive-subset pruning
WP configuration	Alt-Ergo + Z3, 5 s, Typed model	per-obligation prover budget

Table 4: Defaults for execution sampling (§4.3), SFT synthesis (§4.4), reward computation (§4.5), inference, and verification.

Proof. Fix a trace ν rejected by $f(\nu)$ rollouts. In a uniformly random ordering of the group, ν contributes to the marginal gain of exactly one player—the first of those $f(\nu)$ rollouts to appear—and by symmetry each of them is first with probability $1/f(\nu)$. Player i therefore receives expected credit $1/f(\nu)$ for each $\nu \in R_i$ and zero otherwise. Summing over $\nu \in \mathcal{N}$ and normalizing by $|\mathcal{N}|$ gives the closed form; efficiency follows because each covered trace distributes exactly one unit of credit among its $f(\nu)$ rejectors. □

C Implementation Details

Table 4 lists the implemented defaults. SFT synthesis, RL, and inference share one rollout–decompose–pool–filter–compose implementation. RL also filters the union once, then uses clause provenance to assign survivors back to their proposing rollouts before computing credit (Appendix L).

The exact near-input tier order is `0, 1, 2, -1, 3, 5, 8, -2, 10, 17, 4, -3, 25, 6, 40, 7, -5, 13, 50, 9`.

C.1 NEGATIVE-TRACE SAMPLING

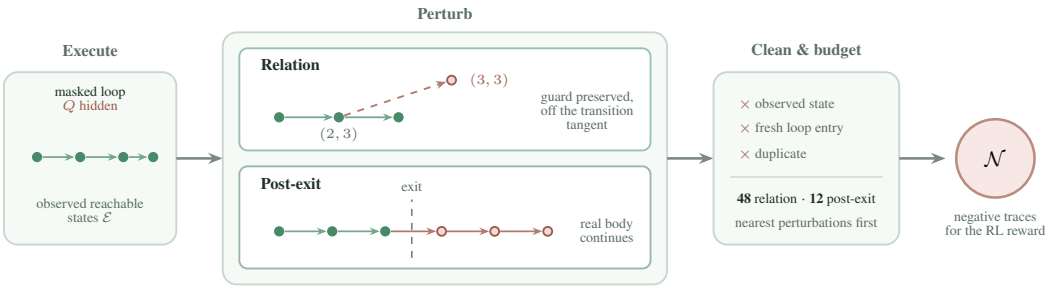

Figure 5: **Construction of target-hidden negative traces.** Concrete executions provide observed reachable states. Two transformations probe relational structure and behavior past a genuine exit. Conservative cleaning and per-family budgets produce $\mathcal{N}$, which is used only for RL reward computation and never exposes Q.

Algorithm 2 summarizes the sampler used for RL. Trace records real loop-head states and, after each genuine exit, continues the real loop body to obtain one post-exit trace. $\mathrm{Perturb}_\Theta$ applies the configured single-variable and pairwise changes; the exact perturbations and budgets are those in Table 4.

The two families are complementary and both are hard by construction. A relation perturbation is admitted only when it stays inside the sampled envelope of its context, preserves the guard truth value, and does not lie on the locally witnessed transition tangent, so only clauses that constrain relations among variables can reject it. A post-exit continuation executes the real body past a witnessed exit into states that no perturbation of an observed state can produce, so only clauses that pin down the exit boundary reject it. Families whose witnesses are rejected by boilerplate bounds (single-variable range escapes and frame equalities) were removed after offline audits. Section 5.6 ablates the structured families against uniform random perturbations.

Algorithm 2 Target-hidden negative-trace sampling

Input: masked loop $\mathcal{L} = (P, B, S)$, input schedule U, configuration Θ
Output: reachable states $\mathcal{E}$, negative traces $\mathcal{N}$

$\mathcal{T}_{\mathrm{exec}}, \mathcal{T}_{\mathrm{post}}, \textit{capped} \leftarrow \mathrm{Trace}(\mathcal{L}, U, \Theta)$ ▷ *capped: some run was capped*
$\mathcal{E} \leftarrow \mathrm{UniqueHeads}(\mathcal{T}_{\mathrm{exec}})$; $M \leftarrow \mathrm{SafeModifiedVars}(S)$
if $M = \varnothing$ or S has unsupported state **then return** $(\mathcal{E}, \varnothing)$
$\mathcal{E}_{\mathrm{sample}} \leftarrow \mathrm{StratifiedSample}(\mathcal{E})$
$C_{\mathrm{rel}} \leftarrow \mathrm{Perturb}_\Theta(\mathrm{Dense}(\mathcal{E}_{\mathrm{sample}}), M)$
if $\neg$*capped* **then** $C_{\mathrm{rel}} \leftarrow C_{\mathrm{rel}} \cup \mathrm{Perturb}_\Theta(\mathrm{Terminals}(\mathcal{T}_{\mathrm{exec}}), M)$
$C_{\mathrm{rel}} \leftarrow \mathrm{TangentAndEnvelopeFilter}(C_{\mathrm{rel}}, \mathcal{T}_{\mathrm{exec}})$
$(C_{\mathrm{rel}}, \mathcal{T}_{\mathrm{post}}) \leftarrow \mathrm{Clean}_{\mathcal{E}}(C_{\mathrm{rel}}, \mathcal{T}_{\mathrm{post}})$
$\widehat{C}_{\mathrm{rel}}, \mathcal{N}_{\mathrm{post}} \leftarrow \mathrm{BudgetAndRoundRobin}(C_{\mathrm{rel}}, \mathcal{T}_{\mathrm{post}}, \Theta)$
$\mathcal{N} \leftarrow \{\langle s \rangle : s \in \widehat{C}_{\mathrm{rel}}\} \cup \mathcal{N}_{\mathrm{post}}$
return $(\mathcal{E}, \mathcal{N})$

$\mathrm{Clean}_{\mathcal{E}}$ removes observed reachable states, possible fresh loop entries, duplicates, and unsupported candidates; a cleaned post-exit continuation remains one trace, and relation candidates become singleton traces. Within each structural bucket the round-robin prefers the smallest counterfactual change, so surviving witnesses sit close to the sampled manifold.

C.2 Group-Relative Optimization Details

For a sampled response group $A_{1:G} \sim \pi_{\theta_{\text{old}}}(\cdot \mid \mathcal{L})$, we normalize Equation 2 within its group,

$$\widehat{\text{Adv}}_i = \frac{r_i - \text{mean}_j\, r_j}{\text{std}_j\, r_j + 10^{-6}},$$

and use the clipped GRPO objective (Shao et al., 2024)

$$\mathcal{J}(\theta) = \mathbb{E}\left[\frac{1}{G}\sum_i \frac{1}{T_i}\sum_u \min\Big(\rho_{i,u}\widehat{\text{Adv}}_i, \text{clip}(\rho_{i,u}, 1-\varepsilon, 1+\varepsilon)\widehat{\text{Adv}}_i\Big) - \beta D_{\text{KL}}(\pi_\theta \| \pi_{\text{ref}})\right]. \quad (3)$$

Here T_i is the token length of response i, $\rho_{i,u}$ is the token-level ratio at token position u to the rollout policy, and π_{ref} is frozen. Clause de-duplication, which forms the clause set scored above, normalizes comparison direction, equality sides, parentheses, immediate commutative operands, harmless identities, and Boolean association, but avoids transformations that require additional arithmetic assumptions. Repeated clauses therefore collapse without affecting the score, and a unique zero-coverage clause costs nothing because it may support another clause or the hidden target. Both coverage and Shapley credit lie in $[0, 1]$, so the full reward lies in $[0, 1.3]$. When the sampler retains no negatives, scoring falls back to binary inductiveness for a nonempty response.

D Generation Prompt

The canonical evaluation template appears below; every target and non-contract comment is stripped before insertion, and general annotation rules are supplied by `system_prompt.txt`. Both training sets are rebuilt with this single prompt pair: archival records are sanitized to the scalar-integer single-loop interface (37,481 RL and 3,200 SFT records retained), and no model-visible prompt contains a target clause or non-contract comment. The released training sets are further curated from this sanitized pool (Appendix G).

Generation prompt

```
You are given a C function. Produce the strongest inductive ACSL loop
invariants that capture the loop's behavior (conservation/relational laws
+ tight progress bounds). Output ONLY invariant lines, each formatted
exactly as:
loop invariant <expr>;
Output at most 20 invariant lines.

Program:
{program}
```

The accompanying system prompt appears verbatim below (also released as `prompt/system_prompt.txt`).

System prompt

```
You are a formal verification expert specializing in ACSL loop invariant
synthesis for Frama-C/WP.

## LOOP INVARIANT DEFINITION

A loop invariant is a property that:
1. Holds before the first loop iteration (initialization).
2. Is preserved by every loop iteration when the loop guard is true
   (inductiveness).
3. Combined with loop exit (`!guard`), helps establish required
   loop-exit facts.

## RULES

- Add one core conservation/relational equality that explains loop
  updates (state transition law).
- Add minimal progress bounds for loop counters and key arithmetic
  variables.
- Prefer strong invariants; avoid weak/tautological ones.
- Do not add unrelated invariants just to increase count.
- MUST NOT modify or rewrite any `unknown()`/`unknownN()` call.
```

```
- `\at(v, Pre)` can ONLY be used with function parameters, never with
  local variables initialized inside the function.
- `\at(v, LoopEntry)` denotes the value of v at the start of the loop
  and can ONLY be used with local variables initialized before the
  loop (using v = unknown();).
- Loop guard is NOT an invariant; weaken strict guards to non-strict
  bounds at exit.
- Parenthesize mixed equality and relational expressions so their
  grouping is explicit.
- Use `loop invariant` only as a line prefix, never inside expressions.
- Emit scalar invariant expressions only; do not declare predicates,
  logic functions, axioms, or lemmas.
- Do NOT use the ternary `? :` operator; split cases with `==>` instead.
- `^` is bitwise XOR in C/ACSL, not exponentiation --- write `x*x*x`
  for x cubed.
- Do NOT place a C boolean directly in arithmetic (`x + (a > b)` is
  invalid); use `==>` to split cases.
- Only use variables declared in the given function.
- Operators: comparison `==` `!=` `<` `<=` `>` `>=`; logical `&&` `||`
  `!` `==>` `<==>`; arithmetic `+` `-` `*` `/` `%`.

## ACSL INVARIANT SYNTAX

Allowed expression forms (all verified by Frama-C/WP):

1. **Bounds**: `0 <= i`, `i <= n`, `x >= 1`
2. **Linear equality**: `i + c == n`, `2 * s == i * (i - 1)`,
   `x == q * y + r`
3. **Polynomial equality**: `x == n * n * n`,
   `6 * s == i * (i + 1) * (2 * i + 1)`
4. **Relational identity** (multi-var algebraic law):
   `1 + x * z - x - z * y == 0`, `p * s - r * q == 1`
5. **Implication**: `i > 0 ==> s >= i`, `a != 0 ==> q >= 1`
6. **Modular**: `i % 5 == 0`, `r >= 0 && r < y`
7. **Bitshift** (power-of-2 only): `p == 1 << i`, `s == (1 << i) - 1`
8. **Conjunction**: `c >= 1 && c <= k`
9. **\at for params**: `n == \at(n, Pre)` (function parameters only)

Forbidden:
- `^` for exponentiation (it is XOR) --- use repeated `*` or `<<` for
  powers of 2.
- `\product`, `\sum`, `\factorial` --- not ACSL scalar operators.
- `? :` ternary --- rewrite with `==>`.
- `\old(x)` --- use `\at(x, Pre)`.
- Type casts `(int)`, `(long)` --- unnecessary in ACSL logic.
- Function calls, predicates, lemmas, axioms --- emit only scalar
  expressions.

Operators (complete list):
  comparison: `==` `!=` `<` `<=` `>` `>=`
  logical:    `&&` `||` `!` `==>` `<==>`
  arithmetic: `+` `-` `*` `/` `%` `<<` `>>`

## OUTPUT

Return ONLY invariant lines, one per line, formatted exactly as:
`loop invariant <property>;`
Do not return code fences, `loop assigns`, the surrounding C function,
or explanations.
```

E TRAINING INTERFACE

The training service is packaged as a self-contained, CPU-only container with gcc, the verifier, Why3, and its prover backend. It exposes `POST /reward`, accepting program source, an array of model responses, `reward_variant` in {`binary`, `whole_coverage`, `base`, `full`}, and `sampler.negative_sampler` in {`random`, `structured`}; the two switches are independent and default to `full` and `structured`. The response reports each rollout's reward together with its coverage and Shapley-credit components. Responses may be raw LLM text, invariant lists, or annotated code; an offline scorer provides the same generation reward over JSONL/Parquet rollout dumps.

F Training–Evaluation Overlap Audit

We compare the final RL (4,992 records and 4,992 distinct program sources) and SFT (33,346 records and 31,865 distinct program sources) training sets against all 832 evaluation programs after target removal. Under three increasingly aggressive normalizations—exact text, alpha-renamed identifiers, and alpha-renaming with integer constants abstracted—no training program matches an evaluation program (Table 5).

We separately measure coarse structural overlap. A structural cell combines the control-flow profile, guard kind, nonlinearity, nondeterminism, and a variable-count band. The evaluation set occupies 122 such cells. RL and SFT share 88 and 41 of them, respectively; their union shares 92 cells. Equivalently, 702 (84.4%), 529 (63.6%), and 754 (90.6%) of the 832 evaluation programs fall in a cell represented by RL, SFT, and their union. These are support statistics, not instance matches. The record-weighted total-variation distances over cells are 0.422 for RL, 0.651 for SFT, and 0.596 for their union, so the training and evaluation distributions are related but not identical. Table 6 gives interpretable feature marginals behind this joint-cell comparison.

	RL	SFT	RL ∪ SFT
Records	4,992	33,346	38,338
Distinct program sources	4,992	31,865	35,170
Exact-text matches	0	0	0
Alpha-renamed matches	0	0	0
Alpha-renamed, constant-abstracted matches	0	0	0
Shared evaluation cells	88 / 122	41 / 122	92 / 122
Evaluation programs in shared cells	702 (84.4%)	529 (63.6%)	754 (90.6%)
Training records in evaluation-supported cells	4,992 (100%)	27,275 (81.8%)	32,267 (84.2%)
TV distance over structural cells	0.422	0.651	0.596

Table 5: Instance and structural-distribution audit of the final training sets against the 832 target-removed evaluation programs. Distribution statistics count training records, matching their sampling frequency.

Feature (% of records/programs)	Evaluation	RL	SFT
Constant guard	24.6	7.5	4.6
Single-variable guard	31.4	31.6	44.0
Variable-vs.-variable guard	37.3	57.3	50.6
Compound guard	6.7	3.7	0.8
Nonlinear	2.5	2.5	10.8
Nondeterministic	28.6	30.3	14.9
≤ 2 variables	35.1	21.2	46.2
3–4 variables	33.9	31.7	12.0
≥ 5 variables	31.0	47.1	41.8

Table 6: Marginal structural distributions of evaluation programs and final training records. The structural-cell TV distances in Table 5 use the joint features rather than these marginals independently.

Scope of the audit. The audit detects direct source reuse under the three reported normalizations; it does not establish semantic inequivalence between arbitrarily rewritten programs or address possible benchmark exposure during foundation-model pretraining. Structural-cell overlap and total-variation distance characterize domain similarity rather than instance-level reuse.

G Training-Pool Curation

The released training sets are filtered and synthesized from the 37,481-record sanitized pool of Appendix D: programs with no training signal and too-easy or too-hard programs are removed, the

selection is aligned with the evaluation distribution over structural cells (control-flow profile, guard kind, nonlinearity, nondeterminism, variable band), and SFT targets are minimal inductive invariant sets synthesized with `gpt-5.6-luna` and verified by Frama-C/WP. Per-stage counts are emitted as machine-readable reports shipped with the code. Table 7 summarizes the released sets.

RL set	4,992 records over 1,131 distinct loop shapes
SFT set	33,346 records over 31,865 distinct program sources
RL–SFT exact-source overlap	1,687 distinct programs

Table 7: Released training sets.

The reward discriminates on the released data. On the 125 released RL programs with a reference answer, the production reward gives the reference a median of 0.845 and its bounds-only projection a median of 0.0; the tautology `1 == 1`, the loop guard copied as an invariant, and an unsound clause score exactly zero on every program. A simulated eight-answer group has a median within-group reward standard deviation of 0.21, so group-relative updates retain a usable advantage signal.

H Experimental Protocol Details

The evaluation workload composition is given in Section 5.1; the 466 Loopy programs are the integer subset of its original 469-program corpus (three floating-point programs excluded), and unsupported inputs remain failures in the common denominator.

Before model invocation, we remove `assert` and `ensures` predicates, executable assertion calls, conventional error-label names, and non-contract comments, including source-provenance paths; preconditions and executable semantics remain visible. CRAFT evaluation runs use the canonical generation template, extractor, semantic deduplication, and Houdini implementation; external tools retain their own orchestration. Each model retains its serving interface and chat template. In CRAFT, API calls set `reasoning_effort=none`; all locally served checkpoints disable thinking. We set temperature 1.0 and top-p 1.0, without an additional top-k, repetition penalty, re-roll, or target feedback. The cap is 8,192 completion tokens for API models, including any provider-internal reasoning tokens, and 2,048 emitted tokens for local vLLM models. Each RQ1 curve reuses the same saved response pool per task.

Training configuration. Tables 8 and 9 record the supplied SFT and RL run configurations. Reward and sampler constants are reported separately in Table 4.

Field	Value
Learning rate	1×10^{-5}
Epochs	3
Batch (per device $\times$ accumulation $\times$ GPUs)	global batch 32; $2 \times 2 \times 8$ or $1 \times 4 \times 8$
Sequence cutoff	2,048
DeepSpeed	ZeRO-3
Scheduler	cosine

Table 8: SFT training configurations.

Field	Value
Batch size	64
Learning rate	1×10^{-6}
KL penalty	`low_var_kl`, coefficient 0.001
Rollouts per prompt	8
Tensor parallelism	1–2
GPU memory utilization	0.3–0.5
Epochs	5
Total steps	200
Checkpoint interval	100

Table 9: RL training configurations.

I Complete Experimental Data

This section collects the per-model and per-stratum measurements behind the main aggregates.

I.1 Probe Curves

Table 10: Complete base-checkpoint probe results by benchmark stratum and sampling budget. Each cell reports a percentage; All / 832 is the whole-workload result. k_{95} is the smallest measured budget at which compose@k reaches 95% of compose@32. Bold marks the best value in each column at a fixed budget.

		Linear / 316		NLA / 50		Loopy / 466		All / 832		
Model	k	pass	comp	pass	comp	pass	comp	pass	comp	k_{95}
Qwen3-1.7B	1	7.6	22.2	2.4	2.0	8.4	25.3	7.8	22.7	32
	4	15.3	33.9	5.0	8.0	15.9	35.2	15.0	33.1	
	8	19.4	37.7	**6.1**	8.0	19.7	39.3	18.8	36.8	
	16	23.2	39.2	**7.3**	10.0	23.5	42.5	22.4	39.3	
	32	26.6	41.5	**8.0**	10.0	27.5	44.6	26.0	41.4	
Qwen3-4B	1	6.2	38.6	0.5	**8.0**	7.2	41.4	6.4	38.3	16
	4	11.7	45.9	1.7	8.0	12.7	50.6	11.6	46.3	
	8	14.1	51.9	3.0	8.0	15.2	55.4	14.1	51.2	
	16	16.6	55.1	4.5	10.0	17.7	59.9	16.5	55.0	
	32	19.3	57.0	6.0	**14.0**	20.4	61.6	19.1	57.0	
Qwen3-8B	1	8.6	43.7	3.2	6.0	8.6	47.9	8.3	43.8	16
	4	16.8	56.6	5.3	6.0	17.4	59.4	16.5	55.2	
	8	**20.7**	60.4	5.9	12.0	22.0	63.1	20.5	59.0	
	16	**24.5**	61.7	6.0	12.0	26.2	66.1	24.4	61.2	
	32	27.8	63.6	6.0	12.0	30.7	67.2	28.1	62.5	
Qwen3-14B	1	8.9	**54.1**	1.1	**8.0**	**11.8**	**56.2**	**10.1**	**52.5**	8
	4	15.7	**64.6**	3.4	**10.0**	**20.2**	**66.1**	**17.5**	**62.1**	
	8	19.3	**68.0**	4.9	12.0	**24.3**	**68.7**	**21.3**	**65.0**	
	16	23.6	**69.3**	5.9	12.0	**28.3**	**70.0**	**25.2**	**66.2**	
	32	**28.2**	**71.5**	6.0	12.0	**32.2**	**71.2**	**29.1**	**67.8**	
Qwen3-30B-A3B	1	**10.6**	48.1	**3.6**	**8.0**	9.8	52.4	9.7	48.1	8
	4	**17.0**	57.6	**5.7**	8.0	17.9	65.0	16.9	58.8	
	8	19.7	61.1	6.0	8.0	21.9	67.2	20.1	61.3	
	16	22.5	62.0	6.0	10.0	26.2	68.2	23.6	62.4	
	32	25.6	62.3	6.0	10.0	30.5	68.5	27.2	62.6	
Llama 3.1 8B	1	0.8	29.4	0.0	2.0	0.9	32.0	0.8	29.2	8
	4	3.0	49.7	0.0	8.0	3.3	51.7	3.0	48.3	
	8	5.5	55.4	0.0	**14.0**	5.9	58.4	5.4	54.6	
	16	9.3	57.0	0.0	**14.0**	9.6	59.9	8.9	56.0	
	32	14.6	57.0	0.0	**14.0**	14.8	60.1	13.8	56.1	

Table 11: Per-GPU decoding throughput of the four backbones used in training experiments, measured with vLLM on A800-80G GPUs (n=32 responses per task, temperature 1.0, `max_tokens=2048`).

Model	tok/s/GPU
Qwen3-4B	7,445
Qwen3-8B	5,833
Qwen3-14B	3,168
Llama 3.1 8B	5,135

I.2 SFT Probe Curves

The SFT checkpoints (three epochs on the final SFT corpus) are evaluated under the same 832-task protocol as the base probe: 32 saved target-hidden responses per task at temperature 1.0, 5-second per-obligation prover budget, and the current prompt.

Table 12: Complete SFT probe results. k_{95} is the smallest measured $k \in \{1, 4, 8, 16, 32\}$ at which compose@k reaches 95% of its value at $k = 32$.

Whole-response pass rates

Model	pass@1	pass@4	pass@8	pass@16	pass@32
Qwen3-4B + SFT	TBD	TBD	TBD	TBD	TBD
Qwen3-8B + SFT	TBD	TBD	TBD	TBD	TBD
Qwen3-14B + SFT	TBD	TBD	TBD	TBD	TBD
Llama 3.1 8B + SFT	TBD	TBD	TBD	TBD	TBD

Clause-composition rates

Model	compose@1	compose@4	compose@8	compose@16	compose@32	k_{95}
Qwen3-4B + SFT	TBD	TBD	TBD	TBD	TBD	TBD
Qwen3-8B + SFT	TBD	TBD	TBD	TBD	TBD	TBD
Qwen3-14B + SFT	TBD	TBD	TBD	TBD	TBD	TBD
Llama 3.1 8B + SFT	TBD	TBD	TBD	TBD	TBD	TBD

Table 13: Per-GPU decoding throughput of the SFT checkpoints, measured with vLLM on A800-80G GPUs (n=100 responses per task, temperature 1.0, `max_tokens=2048`).

Model	tok/s/GPU
Qwen3-4B + SFT	7,445
Qwen3-8B + SFT	5,833
Qwen3-14B + SFT	3,168
Llama 3.1 8B + SFT	5,135

I.3 Reinforcement-Learning Results

Table 14: Complete comparison before and after RL. The Zero and SFT initializations are evaluated under the same decoding and verification configuration.

Checkpoint	Stage	pass@1	compose@1	compose@8	compose@16	compose@32	k_{95}
Zero	before RL	8.3%	43.8%	59.0%	61.2%	62.5%	16
RL-Zero	after RL	6.80%	57.93%	71.27%	73.44%	74.28%	8
SFT	before RL	TBD	TBD	TBD	TBD	TBD	TBD
SFT+RL	after RL	TBD	TBD	TBD	TBD	TBD	TBD

I.4 Ablation Results

The reward variants correspond directly to the two levels of Equation 2. *Binary Inductiveness* assigns one iff the capped, normalized response is nonempty and every clause survives Houdini. *Whole-Rollout Strength* pays the strength score $\mathrm{cov}(C_i^{\mathrm{train}})$ only when every capped clause survives,

$$r_i^{\mathrm{whole}} = \mathbf{1}[C_i^{\mathrm{train}} = \mathrm{set}(\mathrm{dedup}(\bar{A}_i))]\frac{|R_i|}{|\mathcal{N}|}.$$

Clause-Decomposed Strength removes this all-or-nothing indicator and scores the surviving C_i^{train}; the *Full Compositional Credit* additionally adds the group credit $w_g\phi_i$. These are `binary`, `whole_coverage`, `base`, and `full`, respectively, abbreviated Binary, Whole-rollout, Clause-decomposed, and Full in figures and prose. All variants train from the Zero initialization on the curated pool with identical optimization and inference settings.

Table 15: Reward-function ablation on Qwen3-8B, trained from the Zero initialization (%); exact numbers for Figure 4. Best result per column among trained variants in bold.

	pass@k					compose@k				
Variant	1	4	8	16	32	1	4	8	16	32
Zero (untrained reference)	8.3	16.5	20.5	24.4	28.1	43.8	55.2	59.0	61.2	62.5
Binary Inductiveness	9.88	13.33	15.11	17.08	19.30	8.77	15.62	20.19	24.04	27.88
Whole-Rollout Strength	**25.35**	**28.10**	**29.07**	**30.10**	**31.32**	27.76	29.21	29.93	31.13	33.17
Clause-Decomposed Strength	4.60	10.16	13.31	16.27	19.03	**58.29**	65.99	67.79	69.35	70.19
Full Compositional Credit (ours)	6.80	13.42	16.80	20.17	23.44	57.93	**67.19**	**71.27**	**73.44**	**74.28**

The negative-sampler ablation uses matched Qwen3-8B runs from the Zero initialization under the full compositional reward that differ only in the sampler; the Structured row is the Full Compositional Credit run of Table 15. Structured is the sampler of Section 4.3 (relation and post-exit families); Random draws uniform local perturbations under the same trace budget.

Table 16: Negative-sampler ablation on Qwen3-8B, trained from the Zero initialization under the full reward (%). Bold marks the better sampler per column.

	pass@k					compose@k				
Negative sampler	1	4	8	16	32	1	4	8	16	32
Random	**8.63**	**14.40**	**17.00**	19.43	21.82	52.04	60.46	64.54	67.31	68.75
Structured (ours)	6.80	13.42	16.80	**20.17**	**23.44**	**57.93**	**67.19**	**71.27**	**73.44**	**74.28**

Structured sampling improves compose@k by 5.5–6.7 points at every budget, whereas pass@k stays within -1.8 to $+1.6$ points of Random.

I.5 Predictive Validity of Negative Coverage

This audit tests the target-independent strength signal directly rather than inferring its validity only from downstream RL. We collect 6,190 saved candidate invariant sets produced by eight target-hidden generation pipelines on the 832-program test set. For each candidate, the prediction score is its structured relation/post-exit negative coverage, computed without Q; the binary label is the final Frama-C/WP verdict after the untouched target is restored. No target label is used to fit a predictor or choose a threshold.

The continuous analysis contains 5,606 candidates from 752 programs. The remaining 584 candidate rows belong to 80 programs for which the sampler retains no negative group; those rows use the binary reward fallback and have no continuous coverage value, so they are excluded rather than assigned an artificial zero. Because multiple candidate generators are evaluated on the same program,

confidence intervals use 5,000 stratified program-cluster bootstrap replicates: all candidates for a resampled program move together. A second analysis computes AUROC separately within each of the 547 programs having both successful and unsuccessful candidates, then macro-averages over programs. This removes between-program difficulty as an explanation for the ranking.

Population	Candidates	Positive rate	AUROC	AUPRC
All	5,606	0.508	0.738	0.730
Linear	2,336	0.533	0.743	0.768
NLA	400	0.090	0.542	0.132
Loopy	2,870	0.547	0.764	0.778

The pooled AUROC has a program-clustered 95% interval of $[0.711, 0.763]$, and the AUPRC interval is $[0.699, 0.758]$, compared with the pooled positive-rate baseline of 0.508. Within programs, macro AUROC is 0.794, with a clustered interval of $[0.773, 0.814]$; randomly permuting target verdicts among candidates for the same program gives a one-sided $p < 2 \times 10^{-4}$ over 5,000 permutations. Method-stratified AUROCs range from 0.572 to 0.731, so the pooled association is not created by a single candidate generator. Candidates with coverage at least 0.90 verify 78.9% of the time, compared with 29.2% below 0.50.

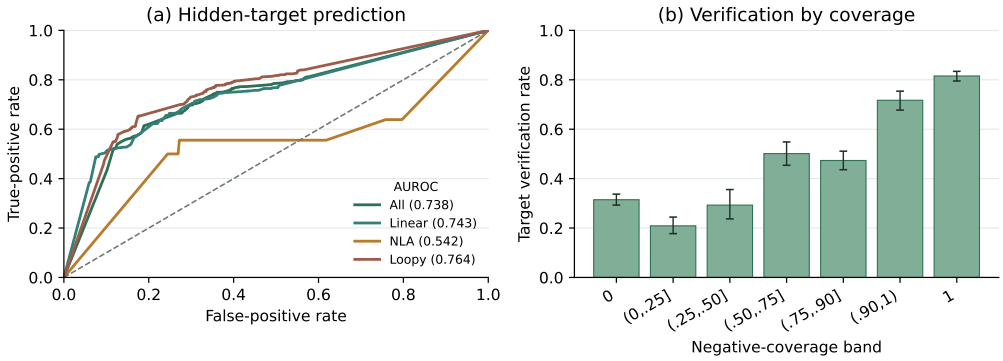

Figure 6: Predictive validity of target-independent negative coverage on saved test-set candidates. (a) ROC curves for restored-target verification; numbers in parentheses are AUROCs. (b) Target verification rate by coverage band, with Wilson 95% intervals. NLA remains close to chance, showing that the current structured negatives discriminate hidden-target utility much better for Linear and Loopy programs than for nonlinear arithmetic.

These results establish association, not causality: coverage is useful for ranking candidate strength but is not itself a proof of relevance to every possible hidden target. In particular, the NLA stratum identifies where richer nonlinear perturbations are needed. The complete deterministic analysis is produced by `scripts/analyze_negative_coverage_auc.py`; machine-readable results are archived in `artifacts/v4/negative_coverage_predictiveness.json`.

I.6 Cross-Model Main Results

Table 17: Complete reasoning-off cross-model results by benchmark stratum and sampling budget. Each cell reports a percentage; All / 832 is the whole-workload result. Rows for each model come from one archived rollout pool of ten responses per task. compose@k uses prefix composition and pass@k uses the unbiased estimator. Bold marks the best value in each column at a fixed budget. GPT-5-mini is excluded because it has no reasoning-off setting.

		Linear / 316		NLA / 50		Loopy / 466		All / 832	
Model	k	pass	comp	pass	comp	pass	comp	pass	comp
GPT-5-nano	1	3.77	34.81	0.40	22.00	3.80	34.55	3.58	33.89
	4	10.79	48.10	1.60	30.00	11.48	51.72	10.62	49.04
	8	15.84	55.38	3.20	44.00	17.58	56.87	16.05	55.53
GPT-5	1	**35.06**	52.22	5.80	36.00	33.78	52.36	32.58	51.32
	4	**47.73**	64.87	8.74	52.00	**50.67**	67.38	**47.03**	65.50
	8	**51.93**	70.25	**9.60**	66.00	**56.13**	73.82	**51.74**	72.00
Claude Sonnet-4.6	1	34.49	**55.38**	**7.80**	**44.00**	**40.67**	**59.44**	**36.35**	**56.97**
	4	40.32	62.97	**8.80**	52.00	47.27	66.31	42.32	64.18
	8	42.87	66.46	**9.60**	64.00	50.12	67.60	44.93	66.95
DeepSeek V4-Flash	1	28.67	52.22	5.00	**44.00**	30.15	51.72	28.08	51.44
	4	42.35	**66.46**	7.52	**60.00**	47.73	**69.96**	43.27	**68.03**
	8	46.64	**71.52**	8.00	**72.00**	53.59	**76.61**	48.21	**74.40**

Each pass@k/compose@k pair at a given k is certified from the same saved k responses.

I.7 Comparison with Specialized Tools

Table 18: Archived target-hidden tool comparison: the `reasoning_effort=none` rows from the main text alongside the archived `reasoning_effort=medium` rerun; Daikon makes no model call and is listed once. Each cell reports the verification rate for Linear, NLA, Loopy, or all 832 programs. The compose@1 rows are the dedicated single-call runs paired with pass@1. Medium-effort results are available only for these single-response measurements; the compose@4 and compose@8 rows use `reasoning_effort=none` and are prefix compositions of the archived ten-rollout pool, as in Table 2.

Method	Reason.	Linear	NLA	Loopy	All	Tokens/task	Time/task
AutoSpec	none	47.78%	6.00%	42.27%	42.19%	2,181.72	27.34 s
AutoSpec	medium	65.82%	8.00%	64.81%	61.78%	25,154.54	54.77 s
SESpec	none	28.80%	4.00%	33.05%	29.69%	22,081.61	68.50 s
SESpec	medium	56.96%	6.00%	49.57%	49.76%	44,807.05	117.31 s
Clause2Inv	none	20.89%	0.00%	0.00%	7.93%	1,056.25	8.41 s
Clause2Inv	medium	19.94%	2.00%	0.00%	7.69%	385.80	29.04 s
Daikon	—	23.10%	0.00%	21.67%	20.91%	0	4.89 s
Loopy	none	20.25%	0.00%	19.74%	18.75%	14,513.37	147.16 s
Loopy	medium	72.78%	12.00%	70.82%	68.03%	111,973.15	381.01 s
Naive	none	15.19%	2.00%	18.88%	16.47%	225.95	2.98 s
Naive	medium	48.73%	8.00%	47.42%	45.55%	4,493.37	28.87 s
CRAFT (pass@1)	none	2.53%	0.00%	4.72%	3.61%	1,135.83	7.86 s
CRAFT (pass@1)	medium	61.08%	14.00%	54.08%	54.33%	6,928.30	53.52 s
CRAFT (compose@1)	none	49.68%	8.00%	45.71%	44.95%	1,135.83	29.66 s
CRAFT (compose@1)	medium	64.24%	14.00%	63.09%	60.58%	6,928.30	66.60 s
CRAFT (compose@4)	none	48.10%	30.00%	51.72%	49.04%	1,905.10	46.40 s
CRAFT (compose@8)	none	55.38%	44.00%	56.87%	55.53%	2,929.47	69.99 s

The main text reports each method's `reasoning_effort=none` row. The shared model-call parameters for the non-reasoning comparison are summarized separately in Table 19.

Table 19: Sampling parameters used in the non-reasoning target-hidden tool comparison.

Method	Sampling budget / task	Temperature / top-p	Completion cap	Reasoning
AutoSpec	8	1.0 / 1.0	8,192	none
SESpec	1 initial + up to 3 refinements per phase	1.0 / 1.0	8,192	none
Clause2Inv	adaptive; 1 per call	1.0 / 1.0	8,192	none
Daikon	no model call	–	–	–
Naive	1	1.0 / 1.0	8,192	none
CRAFT	1, 5, or 2×5	1.0 / 1.0	8,192	none
Loopy	15 initial + up to 7 repairs	0.7 / 1.0	8,192	none

Token totals are means over tasks with at least one model call; accuracy always keeps the 832-task denominator.

I.8 Tool-Specific Behavior under Target Hiding

The evaluated systems interact differently with the target-hidden protocol. The following observations clarify their effective inference procedures rather than attributing performance differences to any single mechanism.

AutoSpec. AutoSpec pools eight parallel proposals in its native orchestration. Its reported result therefore reflects a multi-proposal pipeline rather than a single-response baseline.

Clause2Inv. Clause2Inv normally constructs false-state counterexamples from the postcondition. Because the postcondition is hidden in our protocol, that source of counterexamples is unavailable. Its required transition verification conditions are also unavailable for the 466 Loopy programs; these unsupported instances remain failures in the common denominator.

Loopy. Although 98.8% of Loopy's initial responses contain textual `loop invariant` annotations, its native extractor passes only 1.75% of initial responses to Houdini. This indicates an interface bottleneck between generated text and the clauses accepted by its downstream pipeline.

J Detailed Limitations

Program scope. Our implementation targets numerical programs with one scalar-integer loop. This scope follows the dominant setting in prior loop-invariant benchmarks, but excludes nested loops and invariants over arrays, heaps, or recursively defined predicates. Such invariants require richer representations and proof dependencies, while available training corpora contain too few examples to support the discriminative signal used here. Our state perturbations cannot yet construct reliably informative negatives for inductive predicates or interacting nested loops (Liu et al., 2024a).

Inference engine. In preliminary comparisons, locally evaluated checkpoints achieved lower verification rates under vLLM than under alternative backends despite nominally matched decoding settings. We nevertheless use vLLM for all local base and trained checkpoints because its throughput makes broad sampling practical and its consistent use avoids an additional train–test mismatch. API models retain their provider serving interfaces and chat templates under the explicitly reported decoding and reasoning controls. Local-model success rates may therefore be conservative rather than engine-independent ceilings.

Verifier and specification language. Our filtering and all final judgments use Frama-C/WP, and candidates use ACSL. Parser behavior, generated obligations, prover integration, and syntax affect which clauses survive. The pipeline is not intrinsically tied to this toolchain, but the quantitative conclusions may not transfer unchanged to other verifiers, specification languages, or source languages.

Finite candidate-pool approximation. Before Houdini, the pipeline implementation applies a lightweight AST-based filter to parse, normalize, and deduplicate candidate clauses. This filter is deliberately conservative and does not provide a completeness guarantee for the full ACSL expression language: a semantically useful clause that is not handled by its restricted parser can be omitted. In addition, compose@k caps the candidate union passed to verification at 300 clauses to keep WP cost bounded. When an extreme response pool exceeds this limit, later clauses may be truncated even if they add useful information. Both effects can make the reported compose@k underestimate an ideal unbounded implementation. They affect candidate recall, not the validity of reported successes, because every retained result must still pass the full Frama-C/WP final judge.

Feedback regime. We do not evaluate iterative optimization from verifier feedback. In pilot runs, models quickly reached valid but weak invariants: they passed inductiveness checks, but with Q hidden the verifier did not identify the missing behavioral strength. Refinement therefore yielded little marginal gain, which decreased further as larger initial rollout unions captured the easy corrections. Our results characterize target-independent training and finite-budget composition, not settings with rich counterexamples or target-directed repair.

K Why Thinking Is Disabled for Local Models

For the non-reasoning API experiments reported in the main paper, GPT-family requests set `reasoning_effort=none`, compatible endpoints disable thinking explicitly, and Claude Sonnet disables adaptive thinking with a zero thinking budget. For all locally served checkpoints, including the base probes and models trained in our pipeline, we disable chain-of-thought (CoT) during synthetic data construction, training rollouts, and evaluation. Applying the same setting at all three stages avoids a train–test distribution shift. This choice has two practical motivations.

CoT substantially increases generation cost. Explicit CoT adds reasoning tokens to every rollout, so its cost is multiplied by the sampling budget, while the required output is only a short, machine-parseable set of ACSL clauses.

Invariant clauses compose, but CoTs do not. Invariant output is decomposable: Houdini can remove a failing clause while retaining its siblings, and the survivors compose across rollouts. A CoT has no such clause-level correspondence—correct and incorrect reasoning interleave in one trace, and fragments from different rollouts cannot be merged into one faithful rationale for the composed set. CoT supervision is therefore not aligned with the clause-wise filter-and-compose target.

L Pooled Filtering and Provenance Credit

Let $\bar{A}_i$ be the normalized, capped clause sequence proposed by rollout i, and let $C_i^{\mathrm{pf}} = \mathsf{PF}_{\mathcal{E}}(\mathrm{set}(\bar{A}_i))$. Recall that $\mathsf{Filter}_{\mathcal{L}}(C)$ denotes the implemented filtering pipeline on candidate set C. RL, SFT construction, and inference all merge the candidates and filter them once:

$$C_{\mathcal{G}}^{\mathrm{surv}} = \mathsf{Filter}_{\mathcal{L}}\left(\bigcup_{i=1}^{G} C_i^{\mathrm{pf}}\right). \tag{4}$$

This order allows a clause from one response to be preserved with the help of a companion clause proposed by another response and removes the former training–inference mismatch in Houdini ordering.

RL still requires one scalar per rollout. The implementation retains an owner map from each normalized semantic clause key to every rollout that proposed that key. After Equation (4), it assigns a survivor to all of its owners:

$$C_i^{\mathrm{train}} = \{c \in C_i^{\mathrm{pf}} : \mathrm{key}(c) \in \mathrm{key}(C_{\mathcal{G}}^{\mathrm{surv}})\}. \tag{5}$$

Equivalent spellings therefore share the pooled verdict but retain their own surface forms in rollout diagnostics. Duplicate ownership is intentional: if the same survivor rejects a negative for f rollouts, Equation (1) splits that rejection among the f owners. A supporting clause that rejects no sampled negative receives no direct coverage credit, but it can enable a companion survivor that does; this is the same limitation as any negative-coverage surrogate.

Relation to exact coalition credit. Once the full-group survivor set is fixed, the sets $R_i = \mathrm{rej}(C_i^{\mathrm{train}})$ define an ordinary coverage game, whose exact Shapley value is the closed form in Equation (1). This is not the Shapley value of a more expensive game that re-runs Houdini for every coalition. Even leave-one-out credit under such a set-level value V,

$$\Delta_i = V\left(\mathsf{Filter}_{\mathcal{L}}\left(\bigcup_{j} C_j^{\mathrm{pf}}\right)\right) - V\left(\mathsf{Filter}_{\mathcal{L}}\left(\bigcup_{j\neq i} C_j^{\mathrm{pf}}\right)\right),$$

requires at least $G+1$ union-level Houdini evaluations per prompt and can still miss higher-order interactions; Shapley-style credit over unions requires exponentially many subset evaluations. We instead condition the fixed coverage game on the sampled full group, preserving informative per-rollout variation with one pooled filter call.

Runtime and signal check. We compared this implementation with the historical independent-filtering path on a fixed, stratified sample of 30 archived tasks (10 Linear, 10 NLA, and 10 Loopy), eight rollouts per task, using the same frozen negatives and a 5-second Frama-C/WP budget per obligation. Pooled filtering reduced the mean filter calls from 8.6 to 1.0 per task, total scoring time from 824.39 to 655.60 seconds ($1.26\times$), and median task time from 16.82 to 6.48 seconds ($2.60\times$). Against restored-target per-rollout verdicts, its AUROC was 0.726 versus 0.725 for independent filtering, and average precision was 0.454 for both. Thus the order change improved wall time without an observed loss of target-success discrimination in this diagnostic sample; these 240 rollouts are a validation check, not a substitute for the training ablations.